# Notas de Aula – Redes de Computadores

2026

Estas notas organizam o conteúdo por encontro e combinam o planejamento da disciplina com os registros do quadro. O objetivo é oferecer um roteiro de revisão; os slides e o livro-texto continuam sendo as referências principais.

| Aula | Data | Tema | Cobertura |
|---|---|---|---|
| 1 | 12/08 | Curso e Internet | Organização, avaliação, Internet e protocolos |
| 2 | 14/08 | Borda e acesso | Hosts, AP/BS, redes de acesso e comutação |
| 3 | 19/08 | Núcleo e medição | Atrasos, núcleo da rede e *packet pair* |
| 4 | 21/08 | Bloqueio | Multiplexação estatística, binomial e Poisson |
| 5 | 26/08 | Bloqueio e filas | Caravanas de carros e início de filas |
| 6 | 28/08 | Filas | M/M/1 |
| 7 | 2/09 | Revisão | Teste ao final da aula |
| 8 | 4/09 | Atrasos e medições | Divisão em camadas, segurança |
| 9 | 9/09 | HTTP | Camada de aplicação |
| 10 | 11/09 | DNS | Recursivo e não recursivo |
| 11 | 16/09 | CDN | Arquiteturas P2P e cliente/servidor; sockets; cobrimos dos slides 62 a 110 da 8a edição do livro |
| 12 | 18/09 | Sockets | Sockets e começamos multiplexação vs demux e UDP |
| 13 | 23/09 | Revisão | Revisão, incluindo P2P vs CS |

**AULA 1 | 12/08/2026 – quarta-feira**

# Apresentação do curso e uma primeira visão da Internet

**Cobertura:** introdução ao curso e início do Capítulo 1.

## 1.1 Organização da disciplina

- Professor: Daniel Sadoc Menasché; contato registrado no quadro: `sadoc@ic.ufrj.br`.
- Classroom: avisos, materiais e atividades.
- Planilha da turma: presenças, agenda e trabalhos em grupo.
- A avaliação combina provas, exercícios/minitestes, trabalho em grupo e eventuais bônus.

Se $MP$ é a média das duas maiores notas entre P1, P2 e P3, $MEX$ é a nota dos exercícios/minitestes e $NT$ é a nota do trabalho em grupo, então

$$MF = 0{,}70\,MP + 0{,}15\,MEX + 0{,}15\,NT + \text{bônus}.$$

## 1.2 Roteiro do Capítulo 1

- 1.1 – O que é a Internet?
- 1.2 – A borda da rede (*network edge*).
- 1.3 – O núcleo da rede (*network core*).
- 1.4 – Desempenho: atraso, perda e vazão.
- 1.5 – Pilha de protocolos e modelos de serviço.

## 1.3 O que é a Internet?

### 1.3.1 Visão física

A Internet é uma rede de redes que interliga bilhões de dispositivos. Os dispositivos nas extremidades são chamados **hosts** ou **sistemas finais**; eles se comunicam por enlaces e por equipamentos de comutação, como roteadores e switches.

### 1.3.2 Visão lógica ou de serviço

A Internet também é uma infraestrutura que oferece serviços de comunicação a aplicações distribuídas. As aplicações usam interfaces de programação (APIs) e protocolos para trocar dados sem precisar conhecer todos os detalhes internos da rede.

**Ideia-chave.** A visão física responde “quais elementos compõem a rede?”; a visão de serviço responde “o que a rede oferece às aplicações?”.

## 1.4 Protocolos

Um protocolo é um conjunto de regras que governa a comunicação entre entidades. Ele estabelece:

- a **sintaxe**: formato e ordem dos campos de uma mensagem;
- a **semântica**: significado dos campos e das mensagens, que caracteriza as ações, ou seja, o que fazer ao transmitir ou receber cada mensagem.

## 1.5 Primeira ponte com estatística

Um parâmetro, como a média populacional $\mu$, é desconhecido. A partir de observações $D_1, D_2, \ldots, D_n$, calculamos uma estatística para estimá-lo. A média amostral é

$$\widehat{\mu} = \frac{D_1 + D_2 + \cdots + D_n}{n}.$$

Essa linguagem reaparece ao estimar atraso médio, vazão disponível, perda e capacidade de um caminho.

### Para revisar

1. Dê um exemplo da Internet sob a visão física e outro sob a visão de serviço.
2. Explique a diferença entre sintaxe, semântica e ação em um protocolo.
3. Por que uma média obtida de poucas medições é um estimador, e não necessariamente o valor real do parâmetro?

**AULA 2 | 14/08/2026 – sexta-feira**

# Borda da rede, redes de acesso e compartilhamento de recursos

**Cobertura:** slides até o 21 (inclusive), com ênfase em redes de acesso e comutação.

## 2.1 Borda da rede

Na borda estão os hosts ou sistemas finais: computadores, telefones, servidores, sensores e outros dispositivos que executam aplicações. A **rede de acesso** conecta um sistema final ao primeiro roteador da rede do provedor.

## 2.2 Acesso sem fio: AP e estação-base

- No Wi-Fi, o host se associa a um ponto de acesso (AP), tipicamente seguindo a família IEEE 802.11.
- Em uma rede celular, o host se comunica com uma estação-base (BS), por exemplo em uma rede 5G.
- Nos dois casos existe um trecho de rádio, mas a arquitetura, o alcance, a mobilidade e a gestão dos recursos são diferentes.

Um meio é **guiado** quando o sinal percorre cabo de cobre ou fibra, e **não guiado** quando se propaga pelo ar.

## 2.3 Tecnologias de acesso residencial

- **HFC**: combina fibra na rede do provedor com cabo coaxial no trecho de distribuição; No coaxial, usa-se FDM para dividir a frequência de *upstream* e *downstream* de dados; usuários vizinhos compartilham a mesma frequência de *upstream* por TDMA.
- **DSL**: usa o par de cobre da linha telefônica até a central; o acesso é dedicado nesse trecho físico.
- **Rede doméstica**: normalmente combina modem/ONT, roteador, Ethernet e Wi-Fi.

## 2.4 Comutação de circuitos e de pacotes

Na comutação de circuitos, recursos são reservados antes da transmissão. Na comutação de pacotes, os dados são divididos em pacotes que compartilham enlaces e buffers.

| **Aspecto** | Circuitos | Pacotes |
|---|---|---|
| **Recursos** | Reservados para a conexão | Compartilhados sob demanda |
| **Uso** | Previsível após admissão | Estatístico e variável |
| **Ociosidade** | Recursos podem ficar ociosos | Capacidade aproveitada por outros fluxos |
| **Sobrecarga** | Nova conexão pode ser bloqueada | Podem surgir fila, atraso e perda |

Recursos reservados podem ser separados por frequência (FDM), tempo (TDM) ou códigos (CDMA).

## 2.5 FDM, TDM vs FDMA e TDMA

FDM e TDM são técnicas de **multiplexação**: dividem os recursos de um enlace por tempo ou frequência. FDMA e TDMA são técnicas de **acesso múltiplo**: definem como diferentes usuários ou transmissores compartilham um meio comum.

- **FDM (Frequency Division Multiplexing)**: divide a largura de banda do meio em diferentes faixas de frequência. Cada sinal utiliza uma faixa distinta e as transmissões podem ocorrer simultaneamente.
- **TDM (Time Division Multiplexing)**: divide o acesso ao meio em intervalos de tempo. Cada fluxo utiliza o enlace durante determinados intervalos, compartilhando o mesmo recurso ao longo do tempo.
- **FDMA (Frequency Division Multiple Access)**: aplica a divisão por frequência ao compartilhamento de um meio entre múltiplos usuários. Cada usuário recebe uma faixa de frequência para transmitir.
- **TDMA (Time Division Multiple Access)**: aplica a divisão por tempo ao compartilhamento de um meio entre múltiplos usuários. Cada usuário recebe determinados intervalos de tempo para transmitir.

Assim, a principal diferença está tanto no **recurso dividido** quanto no **objetivo da técnica**: FDM e TDM descrevem como um enlace pode ser multiplexado, enquanto FDMA e TDMA descrevem como múltiplos usuários podem acessá-lo.

| **Técnica** | Recurso dividido | Aplicação principal |
|---|---|---|
| **FDM** | Frequência | Multiplexação de sinais |
| **TDM** | Tempo | Multiplexação de fluxos |
| **FDMA** | Frequência | Acesso de múltiplos usuários |
| **TDMA** | Tempo | Acesso de múltiplos usuários |

No caso de uma rede **HFC**, essas técnicas podem aparecer em conjunto. O espectro disponível no cabo coaxial é dividido por **FDM**, reservando faixas de frequência distintas para *upstream* e *downstream*. Dentro da faixa de *upstream*, que é compartilhada por vários modems, o acesso pode ser organizado por **TDMA**, de modo que diferentes usuários transmitam em intervalos de tempo distintos. Dessa forma, FDM realiza a separação por frequência, enquanto TDMA organiza o compartilhamento temporal da faixa de *upstream*.

### Para revisar

1. Identifique o AP e o roteador de borda em uma rede doméstica.
2. Em que sentido a capacidade de uma rede HFC pode ser compartilhada?
3. Compare a chegada de um novo usuário a um circuito sem capacidade e a um enlace de pacotes congestionado.

**AULA 3 | 19/08/2026 – quarta-feira**

# Atrasos, núcleo da rede e técnica de *packet pair*

**Cobertura planejada:** slides até o 35 (exclusive) e técnica de *packet pair*.

## 3.1 Componentes do atraso

Para um pacote de $L$ bits transmitido por um enlace de taxa $R$ bits/s, o atraso de transmissão é

$$d_{\text{trans}} = \frac{L}{R}.$$

Para um enlace de comprimento $d$ e velocidade de propagação $s$, o atraso de propagação é

$$d_{\text{prop}} = \frac{d}{s}.$$

O primeiro depende do tamanho do pacote e da taxa do enlace; o segundo depende da distância e do meio físico.

## 3.2 Núcleo da rede

- **Encaminhamento**: mover um pacote da interface de entrada para a interface de saída apropriada.
- **Roteamento**: calcular ou aprender os caminhos usados nas decisões de encaminhamento.
- Em *store-and-forward*, o roteador recebe o pacote antes de transmiti-lo no enlace seguinte.
- Se os pacotes chegam mais depressa do que podem sair, surgem fila, atraso de enfileiramento e, com buffer finito, perda.

## 3.3 Técnica de *packet pair*

Dois pacotes de mesmo tamanho $L$ são enviados muito próximos. O gargalo de taxa $R_b$ tende a impor uma dispersão $\Delta$ entre eles:

$$\Delta \approx \frac{L}{R_b} \qquad \Longrightarrow \qquad \widehat{R}_b \approx \frac{L}{\Delta}.$$

Tráfego concorrente, filas, escalonamento e resolução do relógio introduzem erro. A técnica estima principalmente a capacidade do gargalo; capacidade e largura de banda disponível não são sinônimos.

### 3.3.1 Comparação entre *store-and-forward* e *cut-through*

Considere $N$ pacotes de tamanho $L$ atravessando $H$ enlaces idênticos, cada um com taxa $R$. Ignorando filas e processamento, na comutação *store-and-forward* temos

$$T_{\text{SF}} = (N + H - 1)\frac{L}{R} + \sum_{i=1}^{H} d_{\text{prop},i}.$$

O termo $H-1$ aparece porque o primeiro pacote deve ser completamente recebido em cada roteador antes de ser retransmitido.

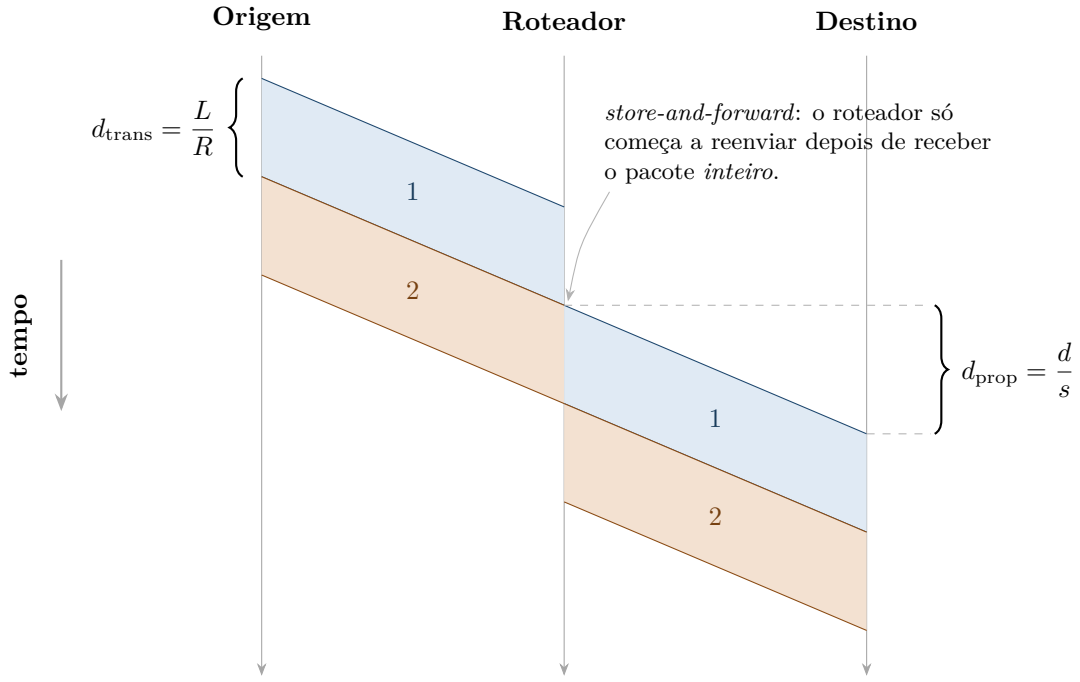



Figure 1: Comutação *store-and-forward*. O roteador somente começa a transmitir um pacote depois de recebê-lo completamente.

#### 3.3.1.1 Comutação *cut-through*

Na comutação *cut-through*, o roteador pode começar a encaminhar um pacote assim que recebe cabeçalho suficiente para determinar a interface de saída.

No caso ideal, com enlaces de mesma taxa e tempo de processamento desprezível,

$$\boxed{T_{\mathrm{CT}} = N\frac{L}{R} + \sum_{i=1}^{H} d_{\mathrm{prop},i}}.$$

O tempo de transmissão $L/R$ é multiplicado pelo número de pacotes $N$, mas não pelo número de enlaces $H$. Os roteadores encaminham os bits como um pipeline contínuo.

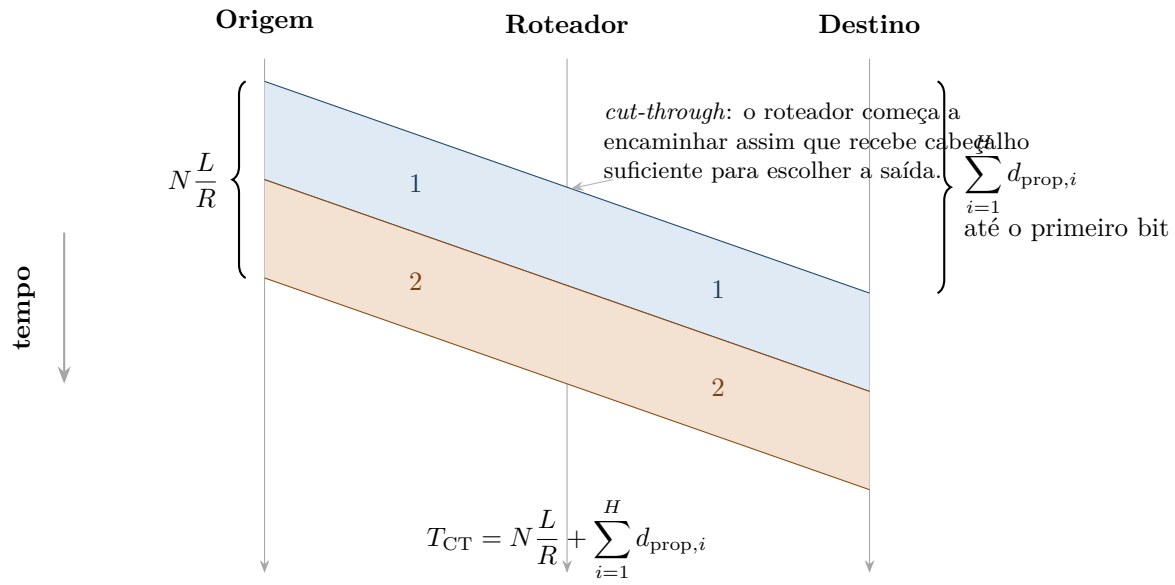



Figure 2: Comutação *cut-through*. No caso ideal, o atraso de transmissão é multiplicado pelo número de pacotes, mas não pelo número de enlaces.

### 3.3.2 Técnica de *packet pair*

Considere dois pacotes de tamanho $L$ enviados consecutivamente por um caminho com dois enlaces. Sejam $R_1$ e $R_2$ as capacidades dos enlaces.

A dispersão entre os pacotes depois do primeiro enlace é

$$\Delta_{\text{in}} = \frac{L}{R_1}.$$

A relação entre $R_1$ e $R_2$ determina se o segundo pacote precisará esperar no roteador.

#### 3.3.2.1 Primeiro enlace mais rápido: $R_1 > R_2$

Neste caso,

$$\frac{L}{R_1} < \frac{L}{R_2}.$$

O pacote 2 chega ao roteador antes que a transmissão do pacote 1 pelo segundo enlace tenha terminado. Portanto, o pacote 2 espera na fila do roteador.

O atraso de fila do segundo pacote é

$$\boxed{W_{q,2} = \frac{L}{R_2} - \frac{L}{R_1}}.$$

Na saída, a dispersão passa a ser

$$\boxed{\Delta_{\text{out}} = \frac{L}{R_2}}.$$

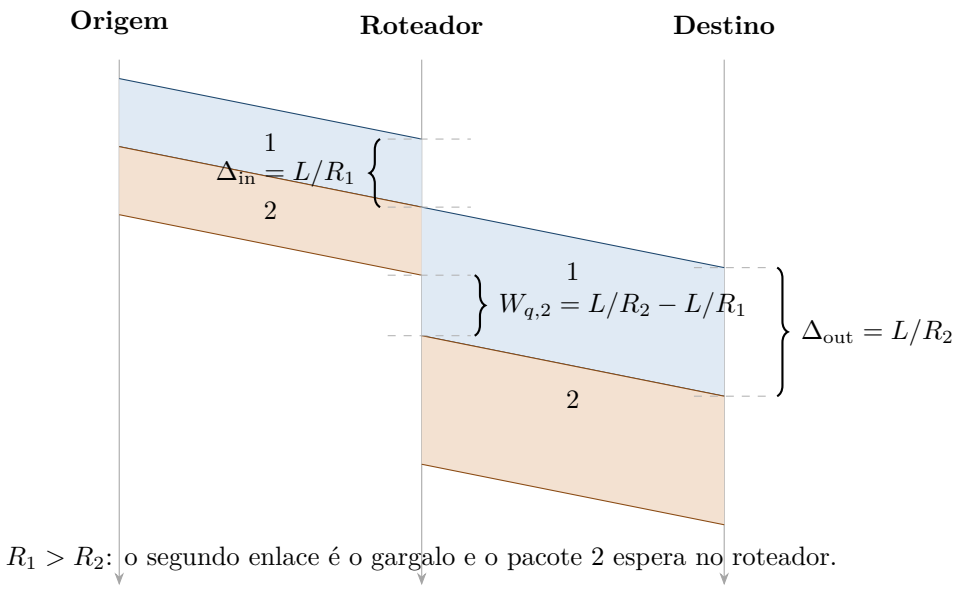



Figure 3: *Packet pair* quando a capacidade do primeiro enlace é maior. O enlace 2 aumenta a dispersão entre os pacotes.

#### 3.3.2.2 Primeiro enlace mais lento: $R_1 < R_2$

Agora,

$$\frac{L}{R_1} > \frac{L}{R_2}.$$

O segundo enlace termina de transmitir o pacote 1 antes da chegada completa do pacote 2. Portanto, não há espera no roteador.

A dispersão observada na saída permanece

$$\boxed{\Delta_{\text{out}} = \frac{L}{R_1}}.$$

Se os dois pacotes ficam prontos simultaneamente na origem, o pacote 2 precisa esperar a transmissão do pacote 1 pelo primeiro enlace:

$$\boxed{W_{q,2}^{(\text{origem})} = \frac{L}{R_1}}.$$

Essa é uma espera na origem, e não uma fila produzida no roteador intermediário.

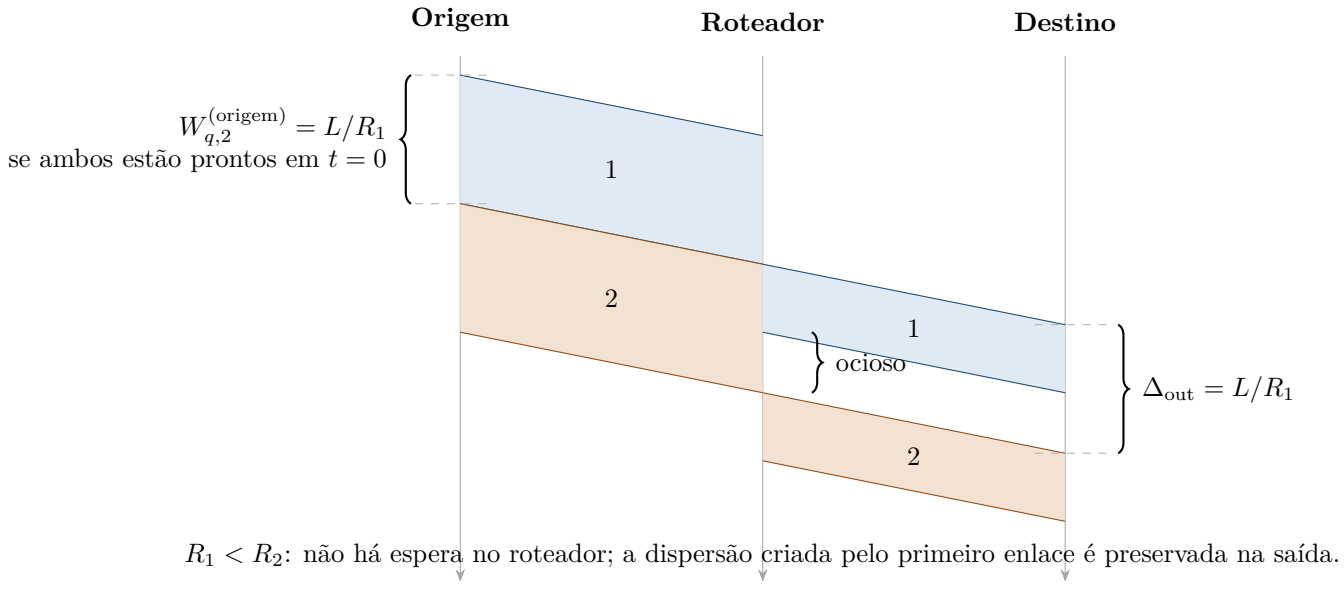



Figure 4: *Packet pair* quando a capacidade do primeiro enlace é menor. O segundo enlace fica temporariamente ocioso entre os pacotes.

#### 3.3.2.3 Interpretação dos dois casos

Os dois exemplos podem ser resumidos por

$$\boxed{\Delta_{\text{out}} = \max\left\{\frac{L}{R_1}, \frac{L}{R_2}\right\} = \frac{L}{\min\{R_1, R_2\}}}.$$

Portanto,

$$\boxed{\widehat{R}_{\text{gargalo}} = \frac{L}{\Delta_{\text{out}}}}.$$

No caso ideal, a dispersão observada entre os dois pacotes revela a capacidade do enlace mais lento. Na prática, tráfego concorrente pode introduzir atrasos adicionais de fila e alterar a dispersão observada.

## Para revisar

1. Diferencie atraso de transmissão e atraso de propagação.
2. Por que o gargalo aumenta a separação entre pacotes consecutivos?
3. Por que é prudente repetir a medição e usar estatísticas robustas?

# Atrasos, *packet pair* e tamanho ótimo de pacote

Notas adicionais – aula de 19 de agosto de 2026

## 3.4 Atraso de transmissão e atraso de propagação

Considere um pacote com $L$ bits e um enlace com taxa $R$ bits/s. O **atraso de transmissão** (ou de serialização) é o tempo necessário para colocar todos os bits do pacote no enlace:

$$d_{\text{trans}} = \frac{L}{R}.$$

Se o enlace possui distância $d$ e o sinal se propaga no meio com velocidade $s$, o **atraso de propagação** é

$$d_{\text{prop}} = \frac{d}{s}.$$

**Diferença essencial.** $L/R$ depende do tamanho do pacote e da taxa do enlace. $d/s$ depende da distância e do meio físico. Um enlace pode ter alta taxa e grande atraso de propagação, como ocorre em enlaces via satélite.

Para um caminho com $H$ enlaces, o atraso fim a fim de um pacote pode ser representado por

$$d_{\text{fim-a-fim}} = \sum_{i=1}^{H} \left( d_{\text{proc},i} + d_{\text{fila},i} + \frac{L}{R_i} + \frac{d_i}{s_i} \right).$$

Neste texto, $H$ é o número de **enlaces** percorridos. Se "$H$ hops" for usado para contar apenas os roteadores intermediários, o caminho terá $H+1$ enlaces e as fórmulas devem ser ajustadas.

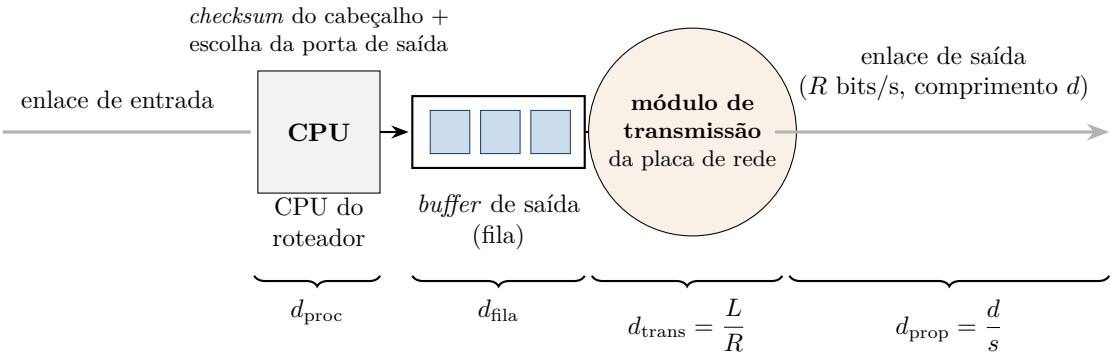



Figure 5: As quatro fontes do atraso nodal, na ordem em que o pacote as encontra. A CPU do roteador verifica o cabeçalho e determina a porta de saída. O pacote é então colocado no *buffer* correspondente. Quando chega à frente da fila, o módulo de transmissão da placa de rede serializa seus $L$ bits à taxa $R$, produzindo o atraso de transmissão $L/R$. Finalmente, os bits se propagam pelo meio físico, produzindo o atraso de propagação $d/s$.

## 3.5 Técnicas de *packet pair*

A técnica de *packet pair* envia dois pacotes de sondagem de mesmo tamanho $L$ muito próximos. Em um gargalo de taxa $R_b$, os pacotes precisam ser serializados. Na situação ideal, a dispersão observada na saída do gargalo é

$$\Delta_{\text{out}} \approx \frac{L}{R_b},$$

de modo que

$$\widehat{R}_b \approx \frac{L}{\Delta_{\text{out}}}.$$

O atraso de propagação comum aos dois pacotes muda o instante absoluto de chegada, mas tende a se cancelar quando se considera apenas a diferença entre os instantes de chegada. Por isso a dispersão pode revelar a taxa de serialização do gargalo.

Na prática, há várias fontes de erro:

- tráfego concorrente pode entrar entre os pacotes e aumentar a dispersão;
- filas e políticas de escalonamento podem comprimir ou expandir o par;
- caminhos diferentes, offloading e resolução limitada do relógio afetam a medição;
- um único par é frágil, razão pela qual se usam repetições, filtros e trens de pacotes.

**Capacidade versus largura de banda disponível.** O *packet pair* clássico procura principalmente a capacidade do gargalo. A capacidade livre depende do tráfego concorrente e exige hipóteses ou técnicas adicionais.

## 3.6 Fragmentação de uma mensagem em pacotes

Considere uma mensagem com $L'$ bytes úteis que atravessa $H$ enlaces idênticos. Nesta seção, para manter as fórmulas em bytes, $R$ representa a taxa em **bytes/s**. Se a taxa do enlace for dada em bits/s, basta usar $R/8$ como taxa em bytes/s, ou multiplicar os numeradores por 8. A mensagem é dividida em $N$ pacotes. Cada pacote contém $\ell$ bytes úteis e um cabeçalho de $h$ bytes. Inicialmente, suponha que $L'$ é múltiplo de $\ell$, de modo que

$$N = \frac{L'}{\ell}.$$

As hipóteses do modelo são:

- comutação *store-and-forward*;
- todos os enlaces têm a mesma taxa $R$;
- não há perdas, retransmissões, filas nem atraso de processamento;
- os pacotes são transmitidos em pipeline;
- o atraso de propagação será acrescentado separadamente.

O primeiro pacote leva

$$H\frac{\ell + h}{R}$$

para ser serializado nos $H$ enlaces. Depois que o pipeline é preenchido, cada um dos $N-1$ pacotes restantes chega separado pelo intervalo $(\ell+h)/R$. Logo,

$$T_{\text{tx}}(\ell) = (H + (N-1))\,\frac{\ell+h}{R} \tag{1}$$

$$= \frac{(H+N-1)(\ell+h)}{R}. \tag{2}$$

Substituindo $N = L'/\ell$,

$$T_{\text{tx}}(\ell) = \frac{1}{R}\left(H - 1 + \frac{L'}{\ell}\right)(\ell+h) \tag{3}$$

$$= \frac{1}{R}\left(L' + (H-1)\ell + \frac{L'h}{\ell} + (H-1)h\right). \tag{4}$$

Se cada enlace também possui atraso de propagação $d/s$, o atraso total, sob as mesmas hipóteses, torna-se

$$T_{\text{total}}(\ell) = T_{\text{tx}}(\ell) + H\frac{d}{s}.$$

O termo de propagação não altera o tamanho ótimo de pacote neste modelo, pois não depende de $\ell$.

## 3.7 Tamanho útil ótimo do pacote

Os termos constantes de (4) não afetam a minimização. Assim, devemos minimizar

$$f(\ell) = (H-1)\ell + \frac{L'h}{\ell}.$$

Derivando,

$$f'(\ell) = (H-1) - \frac{L'h}{\ell^2}.$$

Igualando a zero,

$$(H-1) = \frac{L'h}{\ell^2},$$

e, portanto,

$$\boxed{\ell^* = \sqrt{\frac{L'h}{H-1}}}.$$

A segunda derivada é

$$f''(\ell) = \frac{2L'h}{\ell^3} > 0,$$

logo o ponto estacionário é um mínimo para $H > 1$, $h > 0$ e $L' > 0$.

O número contínuo de pacotes correspondente é

$$\boxed{N^* = \frac{L'}{\ell^*} = \sqrt{\frac{L'(H-1)}{h}}}.$$

Na prática, $N$ deve ser inteiro, o último pacote pode ser menor, e $\ell$ é limitado pela MTU. Devem-se comparar os inteiros próximos de $N^*$ e respeitar os limites do sistema.

## 3.8 A mesma derivação em função de $N$

Como $\ell = L'/N$, a Equação (2) também pode ser escrita como

$$T_{\text{tx}}(N) = \frac{(H + N - 1)(L'/N + h)}{R} \tag{5}$$

$$= \frac{1}{R}\left(L' + (H-1)h + Nh + \frac{(H-1)L'}{N}\right). \tag{6}$$

Derivando a parte não constante,

$$\frac{d}{dN}\left(Nh + \frac{(H-1)L'}{N}\right) = h - \frac{(H-1)L'}{N^2}.$$

Igualando a zero, obtemos novamente

$$N^* = \sqrt{\frac{(H-1)L'}{h}}.$$

## 3.9 Quando a fórmula degenera para $L(N + H - 1)/R$?

Se desprezarmos o cabeçalho, isto é, se $h = 0$, e chamarmos de $L$ o tamanho de cada pacote, então $\ell = L$ e a mensagem total satisfaz

$$L' = NL.$$

Substituindo essas três condições na Equação (4),

$$T_{\text{tx}} = \frac{L' + L(H-1)}{R} \tag{7}$$

$$= \frac{LN + L(H-1)}{R} \tag{8}$$

$$= \boxed{\frac{L(N + H - 1)}{R}}. \tag{9}$$

Portanto, a identificação está correta sob as seguintes condições:

$$h = 0, \qquad \ell = L, \qquad L' = NL,$$

além das hipóteses de enlaces iguais, *store-and-forward*, ausência de filas e pipeline perfeito.

Quando $h = 0$, o modelo ideal favorece pacotes cada vez menores: reduzir $\ell$ melhora o pipeline sem custo adicional. O cabeçalho, o processamento por pacote, a MTU e outros custos reais impedem esse limite irreal e produzem um compromisso entre fragmentação e overhead.

## 3.10 Resumo das expressões

| **Grandeza** | Expressão |
|---|---|
| **Atraso de transmissão de um pacote** | $L/R$ |
| **Atraso de propagação de um enlace** | $d/s$ |
| **Dispersão ideal de *packet pair*** | $\Delta \approx L/R_b$ |
| **Estimativa do gargalo** | $\widehat{R}_b \approx L/\Delta$ |
| **Mensagem fragmentada** | $(H+N-1)(\ell+h)/R$ |
| **Tamanho útil ótimo** | $\ell^* = \sqrt{L'h/(H-1)}$ |
| **Número contínuo ótimo de pacotes** | $N^* = \sqrt{L'(H-1)/h}$ |
| **Caso sem cabeçalho** | $L(N+H-1)/R$ |

**AULA 4 | 21/08/2026 – sexta-feira**

# Multiplexação estatística e probabilidade de bloqueio

**Cobertura:** slide 35 e desenvolvimento, no quadro, do modelo probabilístico de usuários ativos.

## 4.1 Problema de dimensionamento

Considere $m$ usuários que compartilham um recurso capaz de atender simultaneamente $r$ usuários. Cada usuário está ativo com probabilidade $p$. Se $A$ é o número de usuários ativos, então, sob independência e probabilidades iguais,

$$A \sim \text{Binomial}(m, p), \qquad \Pr(A = a) = \binom{m}{a} p^a (1-p)^{m-a}.$$

Além disso,

$$\mathbb{E}[A] = mp.$$

A média é a carga esperada, não um limite máximo para $A$.

## 4.2 Probabilidade de sobrecarga

Há demanda acima da capacidade quando $A > r$:

$$\Pr(A > r) = \sum_{a=r+1}^{m} \Pr(A = a) = 1 - \sum_{a=0}^{r} \Pr(A = a).$$

Um projeto probabilístico escolhe $r$ de modo que

$$\Pr(A > r) \leq \varepsilon,$$

onde $\varepsilon$ é a tolerância de bloqueio, perda ou sobrecarga.

**Cuidado.** Escolher apenas $r = mp$ geralmente não produz uma probabilidade pequena de sobrecarga. É preciso considerar a variabilidade e a cauda da distribuição.

## 4.3 Aproximação de Poisson

Quando $m$ é grande, $p$ é pequeno e $\lambda = mp$ permanece moderado,

$$\Pr(A = a) \approx e^{-\lambda} \frac{\lambda^a}{a!}.$$

## 4.4 Fila e garantias

Na comutação de pacotes, o uso pode ser imediato e a garantia costuma ser probabilística. Na comutação de circuitos, a reserva permite garantia ao usuário aceito, mas uma nova chamada pode ser recusada se todos os circuitos estiverem ocupados.

**Síntese.** A multiplexação estatística aumenta a eficiência porque nem todos os usuários ficam ativos ao mesmo tempo; o preço é um risco controlado de fila, perda ou bloqueio.

### Exercício guiado

Uma rede possui $m = 100$ usuários, cada um ativo com probabilidade $p = 0{,}10$. Assim, $\mathbb{E}[A] = 10$. Para uma capacidade $r$:

1. calcule $\Pr(A > r)$ usando a cauda binomial;
2. compare com a aproximação de Poisson de parâmetro $\lambda = 10$;
3. encontre o menor $r$ que atende a uma meta $\varepsilon$;
4. explique por que $r = 10$ não equivale a uma garantia de baixa sobrecarga.

# Da binomial ao tempo de descongestionamento

Complemento da aula de 21 de agosto de 2026

## 4.5 A mesma binomial em dois sistemas diferentes

Considere $m$ usuários. Em uma escala de tempo adequada, defina

$$I_i = \begin{cases} 1, & \text{se o usuário } i \text{ está ativo;} \\ 0, & \text{caso contrário.} \end{cases}$$

Se os usuários são independentes e $\Pr(I_i = 1) = p$, então

$$A = \sum_{i=1}^{m} I_i \sim \text{Binomial}(m, p).$$

A distribuição matemática é a mesma em comutação de pacotes e de circuitos. O que muda é a definição de "ativo", a escala de observação e a pergunta feita ao modelo.

| | Comutação de pacotes | Comutação de circuitos |
|---|---|---|
| **Escala de tempo** | Slot curto, intervalo de escalonamento ou instante de medição da carga. | Duração de uma chamada/conexão ou instante de admissão de uma nova chamada. |
| **Evento** $I_i = 1$ | O usuário transmite pacotes ou demanda taxa no slot observado. | O usuário mantém uma chamada/conexão que ocupa um circuito. |
| **Variável** $A$ | Número de fontes simultaneamente ativas. | Número de chamadas oferecidas ou circuitos potencialmente ocupados. |
| **Pergunta típica** | Qual é $\Pr(A > r)$, isto é, a chance de a demanda superar a capacidade e produzir fila ou perda? | Qual é a chance de uma nova chamada não encontrar circuito disponível? Ou, antes do controle de admissão, qual é $\Pr(A > r)$ para a demanda oferecida? |
| **Interpretação da garantia** | Probabilística: busca-se manter fila, atraso ou perda abaixo de uma meta. | Determinística para chamadas aceitas; probabilística para o bloqueio na admissão. |

**Pergunta e modelo devem combinar.** Com admissão rígida e apenas $r$ circuitos, o número de circuitos efetivamente ocupados nunca excede $r$. Nesse caso, $\Pr(A > r)$ deve referir-se à *demanda oferecida*, antes da recusa, ou ser substituída diretamente pela probabilidade de bloqueio vista por uma chegada. A binomial é apropriada quando há uma população finita de fontes independentes; o modelo clássico de Erlang usa hipóteses temporais diferentes.

## 4.6 Da binomial à Poisson: derivação heurística

Considere

$$A_m \sim \text{Binomial}(m, p_m), \qquad p_m = \frac{\lambda}{m}.$$

Assim, o número médio de usuários ativos permanece fixo:

$$\mathbb{E}[A_m] = mp_m = \lambda.$$

Para um valor fixo $a$,

$$\Pr(A_m = a) = \binom{m}{a}\left(\frac{\lambda}{m}\right)^a\left(1-\frac{\lambda}{m}\right)^{m-a} \tag{10}$$

$$= \frac{m(m-1)\cdots(m-a+1)}{a!}\frac{\lambda^a}{m^a}\left(1-\frac{\lambda}{m}\right)^m\left(1-\frac{\lambda}{m}\right)^{-a} \tag{11}$$

$$= \frac{\lambda^a}{a!}\underbrace{\prod_{j=0}^{a-1}\left(1-\frac{j}{m}\right)}_{\longrightarrow 1}\underbrace{\left(1-\frac{\lambda}{m}\right)^m}_{\longrightarrow e^{-\lambda}}\underbrace{\left(1-\frac{\lambda}{m}\right)^{-a}}_{\longrightarrow 1}. \tag{12}$$

Quando $m \to \infty$, $p_m \to 0$ e $mp_m = \lambda$, obtemos

$$\boxed{\Pr(A_m = a) \longrightarrow e^{-\lambda}\frac{\lambda^a}{a!}}$$

e escrevemos, aproximadamente,

$$A_m \approx \text{Poisson}(\lambda).$$

**Interpretação.** A aproximação de Poisson é natural quando existem muitas fontes, cada fonte fica ativa com probabilidade pequena, e a carga média total $\lambda = mp$ permanece moderada.

## 4.7 Do estado da rede ao tempo de recuperação

Até aqui, $A$ descreve uma fotografia da rede em um slot. Agora queremos uma variável aleatória temporal: quantos slots serão necessários para que o sistema volte a ficar descongestionado?

Para cada slot futuro $t = 1, 2, \ldots$, seja $A_t$ o número de usuários ativos. Defina

$$q = \Pr(A_t \le r), \qquad 1 - q = \Pr(A_t > r).$$

No modelo binomial,

$$q = \sum_{a=0}^{r}\binom{m}{a}p^a(1-p)^{m-a}.$$

Defina a nova variável aleatória

$$\boxed{S = \min\{s \ge 1 : A_s \le r\}},$$

isto é, $S$ é o número de slots futuros até observarmos o primeiro slot descongestionado.

Se os slots são independentes e têm a mesma probabilidade $q$ de sucesso,

$$S \sim \text{Geom}(q),$$

com suporte $\{1, 2, 3, \ldots\}$. Sua PMF é

$$\Pr(S = s) = (1 - q)^{s-1} q, \qquad s = 1, 2, \ldots,$$

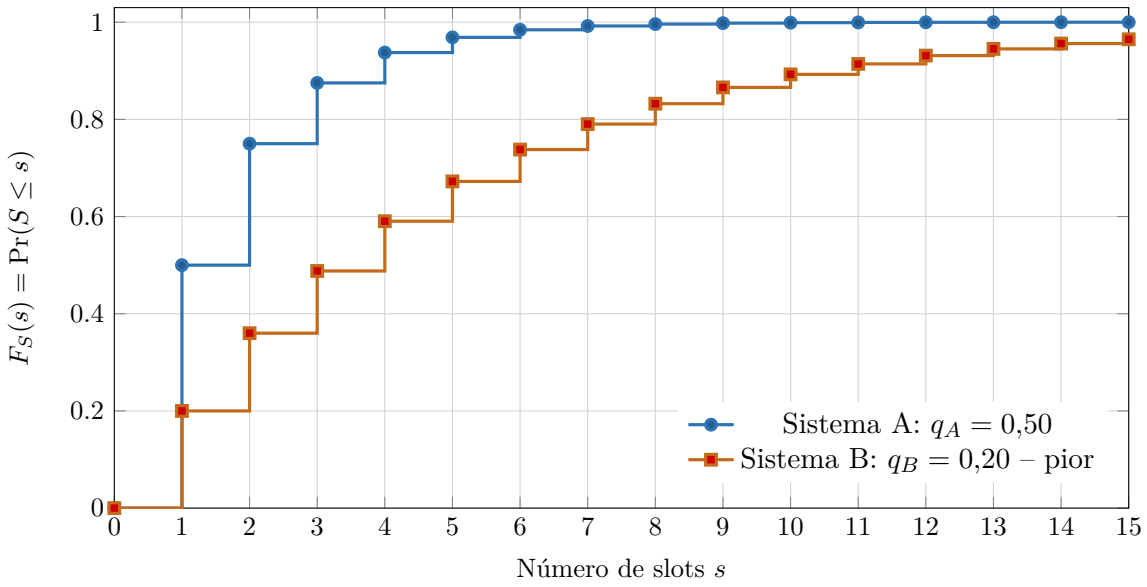
e sua função de distribuição acumulada (CDF) é

$$F_S(s) = \Pr(S \leq s) = 1 - (1 - q)^s, \qquad s = 1, 2, \ldots.$$

**Hipótese forte.** Se usuários permanecem ativos por vários slots, os valores $A_t$ são correlacionados e $S$ não é necessariamente geométrica. A geométrica é o modelo básico obtido sob independência entre slots.

## 4.8 Duas CDFs para o tempo de descongestionamento

O gráfico compara dois sistemas. No Sistema A, um slot está descongestionado com probabilidade $q_A = 0{,}50$; no Sistema B, essa probabilidade é apenas $q_B = 0{,}20$.




**Pergunta aos alunos.** Qual sistema você preferiria? Para um mesmo valor de $s$, qual curva oferece maior probabilidade de já ter saído do congestionamento?

**Leitura do gráfico.** O Sistema A é preferível: sua CDF sobe mais cedo. A curva do Sistema B está mais à direita e é pior, pois atribui maior probabilidade a tempos longos de recuperação. Em termos de ordem estocástica, $S_A$ é estocasticamente menor que $S_B$ (o que pode confundir com o fato de a CDF do sistema $S_A$ estar acima da CDF do sistema $S_B$).

## 4.9 Como caracterizar uma variável aleatória

Uma variável aleatória pode ser descrita de várias maneiras equivalentes ou complementares:

- pela **CDF**, $F_S(s) = \Pr(S \leq s)$, que sempre caracteriza a distribuição;
- pela **PMF**, $p_S(s) = \Pr(S = s)$, quando a variável é discreta;
- por uma **transformada**, como a função geradora de probabilidades ou a função geradora de momentos;
- por **todos os momentos**, quando a distribuição é determinada por seus momentos.

Para a geométrica com suporte começando em 1, a função geradora de probabilidades é

$$G_S(z) = \mathbb{E}[z^S] = \sum_{s=1}^{\infty} z^s (1-q)^{s-1} q = \frac{qz}{1-(1-q)z}.$$

## 4.10 Média da geométrica pela falta de memória

A geométrica possui a propriedade de falta de memória:

$$\Pr(S > m + n \mid S > m) = \Pr(S > n).$$

Depois de um slot congestionado, a situação probabilística para os slots futuros é igual à situação inicial.

Seja

$$\mu = \mathbb{E}[S].$$

No primeiro slot, ocorrem duas possibilidades:

- com probabilidade $q$, o sistema descongestiona imediatamente e $S = 1$;
- com probabilidade $1 - q$, gastamos um slot congestionado e, pela falta de memória, o tempo médio restante volta a ser $\mu$.

Portanto,

$$\mu = q \cdot 1 + (1-q)(1+\mu) \tag{13}$$
$$= 1 + (1-q)\mu. \tag{14}$$

Reorganizando,

$$q\mu = 1,$$

e concluímos que

$$\boxed{\mathbb{E}[S] = \frac{1}{q}}.$$

Nos dois sistemas do gráfico,

$$\mathbb{E}[S_A] = \frac{1}{0{,}50} = 2 \text{ slots}, \qquad \mathbb{E}[S_B] = \frac{1}{0{,}20} = 5 \text{ slots}.$$

Isso confirma numericamente a leitura visual: a curva mais à direita é a pior.

**Fechamento da aula.** A binomial descreve quantos usuários estão ativos em uma fotografia da rede; a Poisson oferece uma aproximação para muitas fontes raramente ativas; e a geométrica descreve, sob independência temporal, quanto tempo esperamos até o sistema voltar a ficar descongestionado.

## Referências de acompanhamento

- J. F. Kurose e K. W. Ross, *Computer Networking: A Top-Down Approach*, Capítulo 1.
- Slides oficiais dos autores: https://gaia.cs.umass.edu/kurose_ross/ppt.php.
- Agenda e materiais da disciplina disponíveis nos canais informados em aula.

**AULA 5 | 26/08/2026 – quarta-feira**

# Do pipeline à teoria de filas

**Cobertura:** slides 36 até slide 52 sobre teoria de filas.

**Roteiro da aula.** Primeiro retomamos o envio de vários pacotes por um caminho com múltiplos enlaces. Depois comparamos *store-and-forward* e *cut-through*. Finalmente, introduzimos um modelo de fila para relacionar taxa de chegada, capacidade, utilização, número de pacotes e atraso.

## 5.1 Retomada: pacotes em pipeline

Considere $N$ pacotes de mesmo tamanho $L$ bits atravessando $H$ enlaces. Todos os enlaces têm taxa $R$ bits/s e os nós usam *store-and-forward*. Por enquanto, ignore filas, perdas, processamento e propagação.

O tempo de transmissão de um pacote em um enlace é

$$\tau = \frac{L}{R}.$$

O primeiro pacote precisa ser completamente transmitido nos $H$ enlaces e chega após $H\tau$. A partir daí, graças ao pipeline, chega um novo pacote a cada intervalo $\tau$. Portanto,

$$\boxed{T_{N,H} = (H + N - 1)\frac{L}{R}}.$$

O termo $N + H - 1$ pode ser visualizado como $H$ intervalos para o primeiro pacote e $N - 1$ intervalos para esvaziar o restante do pipeline.

Se cada enlace também possui atraso de propagação $d_i/s_i$, ele é somado uma vez por enlace:

$$T_{N,H} = (H + N - 1)\frac{L}{R} + \sum_{i=1}^{H} \frac{d_i}{s_i}.$$

Observe que a propagação não é multiplicada por $N$: os pacotes também se propagam em pipeline.

**Verificação.** Se $N = 1$, obtemos $HL/R$. Se $H = 1$, obtemos $NL/R$. Por que esses dois casos-limite fazem sentido?

## 5.2 Comutação e o momento de encaminhar

O nó intermediário pode decidir quando começar a transmitir o pacote pelo próximo enlace.

| | *Store-and-forward* | *Cut-through* |
|---|---|---|
| **Quando encaminha?** | Depois de receber o pacote completo. | Assim que recebe cabeçalho suficiente para determinar a saída. |
| **Memória no nó** | Armazena o pacote inteiro. | Pode usar um buffer menor, embora ainda possa haver filas. |
| **Erro no final do pacote** | Pode ser detectado antes do encaminhamento. | O nó pode já ter propagado parte de um quadro corrompido. |
| **Atraso ideal** | Cada enlace acrescenta aproximadamente $L/R$. | A serialização pode se sobrepor entre enlaces; após o início, o pacote pode atravessar os nós quase como um fluxo de bits. |

**Não confundir conceitos.** *Packet switching* versus *circuit switching* descreve como os recursos são compartilhados. *Store-and-forward* versus *cut-through* descreve quando um nó começa a encaminhar os bits. São duas classificações diferentes.

Em uma rede de pacotes, o encaminhamento é feito pacote a pacote com uma tabela de encaminhamento. Em uma rede de circuitos, há antes uma fase de estabelecimento do circuito e os recursos podem ser reservados durante a conexão. A análise probabilística continua útil nos dois casos, mas em escalas distintas: slots ou rajadas para pacotes; duração de chamadas para circuitos.

## 5.3 A analogia da caravana

A "caravana", traduzida na edição nacional de Kurose & Ross como "comboio de carros", ilustra de forma intuitiva a diferença física e matemática entre o **atraso de transmissão** e o **atraso de propagação**.

Nesta versão da analogia, consideramos um único pacote suficientemente longo para mostrar claramente a diferença entre os regimes *store-and-forward* e *cut-through*.

### 5.3.1 Os elementos da analogia

Imagine uma rodovia com três cabines de pedágio, separadas por trechos de 100 km:

- **Carros individuais** $\longrightarrow$ os **bits**;
- **O comboio de 15 carros** $\longrightarrow$ um **pacote** de dados;
- **As cabines de pedágio** $\longrightarrow$ os **roteadores ou comutadores**;
- **O tempo de atendimento de um carro** $\longrightarrow$ o tempo necessário para transmitir um bit;
- **Os trechos de rodovia** $\longrightarrow$ os **enlaces** de comunicação;
- **A velocidade dos carros** $\longrightarrow$ a velocidade de propagação do sinal no meio físico.

Adotaremos os seguintes parâmetros:

- distância entre dois pedágios: 100 km;
- velocidade dos carros: 1.000 km/h;
- tempo de atendimento: 1 minuto por carro;
- tamanho do comboio: 15 carros.

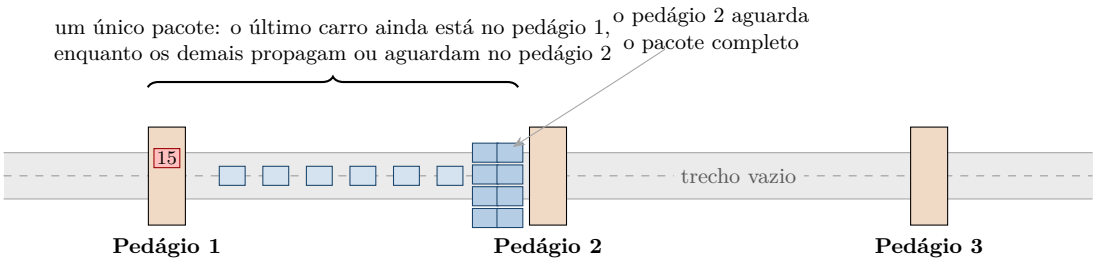
### 5.3.2 Um mesmo pacote nos dois regimes

A Figura 6 representa o sistema no instante $t = 14$ minutos.

No regime *store-and-forward*, o pedágio 2 ainda não recebeu o comboio completo e, portanto, não pode liberar nenhum carro para o trecho seguinte.

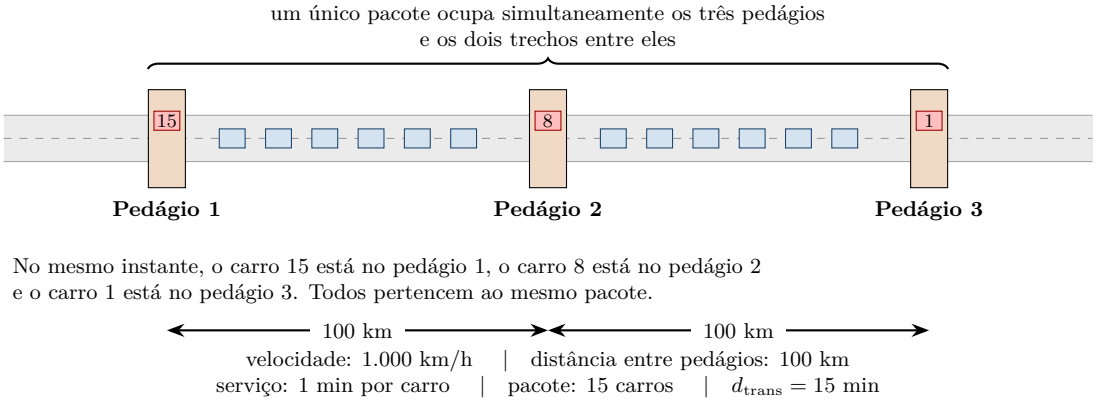
No regime *cut-through*, cada pedágio libera os carros à medida que eles chegam. Nesse mesmo instante, o carro 15 está no pedágio 1, o carro 8 está no pedágio 2 e o carro 1 está no pedágio 3.

**(a) Store-and-forward no instante $t = 14$ min**




No instante $t = 14$, o carro 15 está no pedágio 1, os carros 9–14 ainda propagam e os carros 1–8 já estão armazenados no pedágio 2. O trecho posterior permanece vazio.

**(b) Cut-through no instante $t = 14$ min**




No mesmo instante, o carro 15 está no pedágio 1, o carro 8 está no pedágio 2 e o carro 1 está no pedágio 3. Todos pertencem ao mesmo pacote.

Figure 6: Um único pacote de 15 carros observado no instante $t = 14$ minutos. Em **(a)**, no regime *store-and-forward*, o carro 15 está sendo atendido pelo pedágio 1, os carros 9–14 ainda estão em propagação e os carros 1–8 já chegaram ao pedágio 2, onde permanecem armazenados verticalmente. O pedágio 2 não pode liberar nenhum carro antes de receber o pacote completo e, por isso, o trecho posterior permanece vazio. Em **(b)**, no regime *cut-through*, os carros 15, 8 e 1 estão sendo atendidos simultaneamente pelos pedágios 1, 2 e 3, respectivamente, enquanto os demais carros propagam pelos enlaces.

### 5.3.3 Cálculo dos atrasos básicos

**Atraso de transmissão.** Cada pedágio atende um carro por minuto. Portanto, sua taxa de serviço é

$$R_{\text{pedágio}} = 1 \text{ carro/min.}$$

Como o pacote contém 15 carros, o tempo necessário para transmitir o pacote completo é

$$d_{\text{trans}} = \frac{15 \text{ carros}}{1 \text{ carro/min}} = 15 \text{ minutos.} \tag{15}$$

Na analogia com redes, os 15 carros representam os bits do pacote e a taxa de um carro por minuto representa a taxa de transmissão do enlace.

**Atraso de propagação.** Os carros deslocam-se a 1.000 km/h e os pedágios estão separados por 100 km. Logo,

$$d_{\text{prop}} = \frac{100 \text{ km}}{1.000 \text{ km/h}} \tag{16}$$

$$= 0{,}1 \text{ h} \tag{17}$$

$$= 6 \text{ minutos.} \tag{18}$$

Observe que o atraso de propagação depende da distância e da velocidade, mas não depende do número de carros.

**Tempo para o pacote chegar completamente ao pedágio seguinte.** No regime *store-and-forward*, o último carro deixa o primeiro pedágio após 15 minutos e ainda precisa percorrer 100 km.

Consequentemente, o instante em que o pacote está completamente disponível no pedágio seguinte é

$$d_{\text{trans}} + d_{\text{prop}} = 15 + 6 = 21 \text{ minutos.} \tag{19}$$

### 5.3.4 Funcionamento no regime *store-and-forward*

No regime *store-and-forward*, cada pedágio precisa receber o pacote completo antes de começar a encaminhá-lo:

1. O pedágio 1 atende os 15 carros, um de cada vez.
2. O último carro deixa o pedágio 1 em $t = 15$ minutos.
3. Depois de percorrer 100 km, o último carro chega ao pedágio 2 em

$$t = 15 + 6 = 21 \text{ minutos.}$$

4. Somente em $t = 21$ minutos o pedágio 2 pode começar a atender o primeiro carro.
5. O mesmo procedimento se repete no pedágio 3.

**Ideia central:** o pedágio seguinte pode receber alguns carros antes dos demais, mas não pode encaminhar nenhum deles antes de receber o comboio completo. Isso corresponde a um roteador *store-and-forward*, que somente inicia a transmissão depois de receber o pacote inteiro.

### 5.3.5 O instante $t = 14$ minutos

O instante $t = 14$ minutos permite visualizar claramente a diferença entre os dois regimes.

No pedágio 1, cada carro $i$ termina seu atendimento no instante

$$t = i \text{ minutos.}$$

Como o tempo de propagação é de seis minutos, o carro $i$ chega ao pedágio 2 no instante

$$t = i + 6.$$

Assim, no instante $t = 14$:

- o carro 15 está sendo atendido pelo pedágio 1;
- o carro 8, que deixou o pedágio 1 em $t = 8$, chega ao pedágio 2;
- o carro 1 já chegou ao pedágio 2 desde $t = 7$.

No regime *store-and-forward*, o pedágio 2 precisa esperar até $t = 21$ minutos, quando chega o carro 15. Portanto, em $t = 14$ nenhum carro foi encaminhado para além do pedágio 2.

### 5.3.6 Funcionamento no regime *cut-through*

No regime *cut-through*, o pedágio não espera pela chegada do pacote completo. Cada carro é atendido assim que chega.

O carro 1 segue a seguinte trajetória:

1. é atendido pelo pedágio 1 entre $t = 0$ e $t = 1$;
2. percorre o primeiro trecho entre $t = 1$ e $t = 7$;
3. é atendido pelo pedágio 2 entre $t = 7$ e $t = 8$;
4. percorre o segundo trecho entre $t = 8$ e $t = 14$;
5. chega ao pedágio 3 em $t = 14$.

O carro 8 deixa o pedágio 1 em $t = 8$ e chega ao pedágio 2 em $t = 14$.

O carro 15 começa a ser atendido pelo pedágio 1 em $t = 14$.

Portanto, no instante $t = 14$ minutos,

| | |
|---|---|
| pedágio 1: | carro 15, |
| pedágio 2: | carro 8, |
| pedágio 3: | carro 1. |

Os três pedágios estão simultaneamente ocupados por carros pertencentes ao mesmo pacote.

**Pipeline:** o *cut-through* permite que diferentes partes do mesmo pacote sejam processadas simultaneamente em diferentes estágios. O pacote não precisa estar completamente armazenado em um roteador para que sua parte inicial avance.

### 5.3.7 Atraso fim a fim nos dois regimes

Considere um caminho com $H = 3$ pedágios, seguido por três trechos de propagação. Cada trecho apresenta atraso de propagação de seis minutos.

#### 5.3.7.1 *Store-and-forward*

Em cada estágio, é necessário:

- atender os 15 carros: 15 minutos;
- percorrer o trecho seguinte: 6 minutos.

Portanto,

$$\begin{aligned} T_{\text{SF}} &= H\,(d_{\text{trans}} + d_{\text{prop}}) && (20)\\ &= 3(15+6) && (21)\\ &= \boxed{63 \text{ minutos}}. && (22) \end{aligned}$$

#### 5.3.7.2 *Cut-through*

No regime *cut-through*, os carros formam um pipeline através dos três pedágios.

Com $C = 15$ carros, $H = 3$ pedágios e tempo de serviço por carro $t_c = 1$ minuto, o tempo de serviço do pipeline é

$$(C + H - 1)t_c.$$

Somando os três atrasos de propagação,

$$\begin{aligned} T_{\text{CT}} &= (C+H-1)t_c + Hd_{\text{prop}} && (23)\\ &= (15+3-1)(1) + 3(6) && (24)\\ &= 17 + 18 && (25)\\ &= \boxed{35 \text{ minutos}}. && (26) \end{aligned}$$

A diferença fundamental é que, no regime *store-and-forward*, o tempo de transmissão do pacote completo é pago em cada estágio:

$$3(15) = 45 \text{ minutos}.$$

No regime *cut-through*, o tempo dominante de transmissão é pago uma única vez:

$$15 \text{ minutos},$$

acrescido apenas do pequeno custo de preenchimento dos dois estágios adicionais:

$$(H-1)t_c = 2 \text{ minutos.}$$

### 5.3.8 *Store-and-forward* versus *cut-through*

Os dois regimes diferem no momento em que o equipamento decide encaminhar o pacote:

- ***Store-and-forward*:** o comutador recebe o pacote inteiro, verifica o campo de detecção de erros e somente então inicia a transmissão. Em um caminho com $H$ enlaces idênticos, o atraso de transmissão do pacote é aproximadamente

$$H\frac{L}{R}.$$

- ***Cut-through*:** o comutador começa a encaminhar assim que lê cabeçalho suficiente para identificar a porta de saída. No modelo ideal, o atraso associado ao tamanho completo do pacote é aproximadamente

$$\frac{L}{R},$$

  e não $HL/R$. Existe apenas um pequeno custo adicional para o preenchimento do pipeline.

**O preço do *cut-through*.** Como o quadro é encaminhado antes de terminar de chegar, o comutador pode não conseguir verificar previamente todo o campo de detecção de erros. Quadros corrompidos ou truncados podem ser propagados adiante.
Além disso, o ganho diminui se a porta de saída estiver ocupada ou se sua taxa for menor do que a taxa de entrada. Nesses casos, o equipamento precisa armazenar parte ou todo o quadro. Por isso, o *cut-through* e sua variante *fragment-free*, que examina os primeiros 64 bytes, aparecem principalmente em *switches* Ethernet e em redes de datacenter sensíveis à latência. Os roteadores da Internet operam predominantemente em regime *store-and-forward*.

## 5.4 Da transmissão determinística à fila

Até agora supusemos que o pacote encontra cada enlace livre. Na prática, vários pacotes disputam uma mesma saída. Modele uma interface por um servidor com uma fila: Use a seguinte notação:

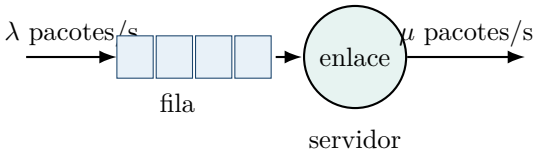



Figure 7: Modelo de fila para um enlace de comunicação. Os pacotes chegam à taxa média $\lambda$, esperam na fila quando o enlace está ocupado e são transmitidos pelo servidor à taxa média $\mu$.

- $\lambda$: taxa média de chegada, em pacotes/s;
- $L$: tamanho médio dos pacotes, em bits;
- $R$: taxa do enlace, em bits/s;
- $1/\mu = L/R$: tempo médio de serviço de um pacote;
- $\mu = R/L$: taxa média de serviço, em pacotes/s.

A intensidade de tráfego ou carga oferecida é

$$\boxed{\rho = \frac{\lambda}{\mu} = \frac{\lambda L}{R}}.$$

Quando o sistema é estável e não há perda de trabalho, $\rho$ também é a fração de tempo em que o enlace permanece ocupado.

Uma justificativa por conservação de taxa é simples. Se o enlace está ocupado durante a fração $p_{\text{ocup}}$ do tempo, sua taxa média de saída é

$$0 \cdot (1 - p_{\text{ocup}}) + \mu p_{\text{ocup}}.$$

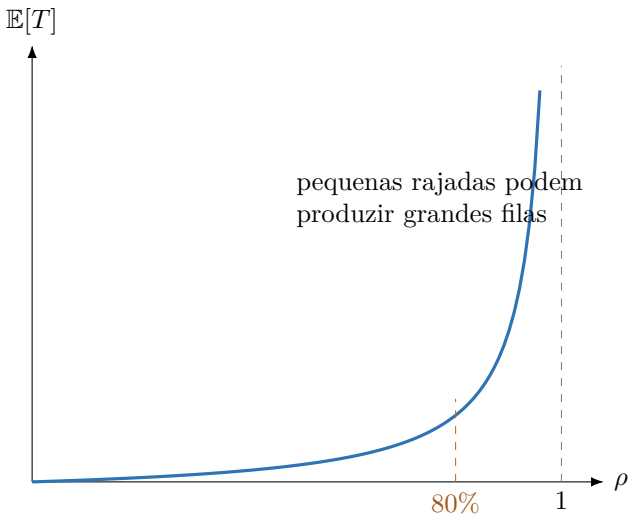
Em regime estacionário, a taxa média de saída deve igualar a taxa de entrada:

$$\lambda = \mu p_{\text{ocup}} \quad \Longrightarrow \quad p_{\text{ocup}} = \frac{\lambda}{\mu} = \rho.$$

**Condição de estabilidade.** Em uma fila com buffer ilimitado, precisamos de $\rho < 1$, isto é, $\lambda < \mu$. Se $\rho > 1$ por tempo prolongado, chega trabalho mais rapidamente do que o enlace consegue atendê-lo e a fila cresce sem limite. Em um buffer finito, o efeito aparece como perdas.

## 5.5 Utilização alta não significa atraso pequeno

Uma utilização próxima de 100% parece eficiente, mas deixa pouca folga para absorver as flutuações aleatórias das chegadas e dos tamanhos dos pacotes.




O formato acima é qualitativo: o crescimento exato depende do modelo de chegadas, do tamanho dos pacotes, do escalonamento e do tamanho do buffer. A mensagem essencial é que o atraso cresce rapidamente quando $\rho$ se aproxima de 1.

**Caso particular $M/M/1$.** Somente se as chegadas forem Poisson, os tempos de serviço forem exponenciais, houver um único servidor e buffer ilimitado, vale

$$\mathbb{E}[T] = \frac{1}{\mu - \lambda}, \qquad \mathbb{E}[W_q] = \frac{\rho}{\mu - \lambda}.$$

Essas fórmulas não são consequências apenas de conhecer $\lambda$ e $\mu$.

## 5.6 Lei de Little

Considere todos os pacotes presentes no sistema: os que esperam na fila e o que está sendo transmitido. Defina

- $\overline{N}$: número médio de pacotes no sistema;
- $\overline{T}$: tempo médio de um pacote no sistema;
- $\lambda$: taxa média efetiva de entrada no sistema.

A Lei de Little estabelece que

$$\boxed{\overline{N} = \lambda\,\overline{T}}.$$

A intuição é uma contagem de "pacote-segundos". Em um intervalo longo de duração $t$, a área sob a curva $N(u)$ é aproximadamente $t\overline{N}$. A mesma área é a soma dos tempos que aproximadamente $\lambda t$ pacotes passaram no sistema, isto é, $(\lambda t)\overline{T}$. Igualando as duas contagens,

$$t\overline{N} = (\lambda t)\overline{T} \quad \Longrightarrow \quad \overline{N} = \lambda\overline{T}.$$

A lei também pode ser aplicada apenas à fila:

$$\overline{N}_q = \lambda\,\overline{W}_q,$$

onde $\overline{N}_q$ exclui o pacote em serviço e $\overline{W}_q$ conta apenas o tempo de espera. É importante combinar corretamente a região contada e o tempo correspondente.

**Exemplo.** Um enlace recebe em média 200 pacotes/s e cada pacote passa em média 15 ms no sistema. Quantos pacotes há, em média, na fila mais o servidor?

$$\overline{N} = 200 \times 0{,}015 = 3 \text{ pacotes.}$$

## 5.7 Síntese e questões de revisão

Ao final da aula, deve ser possível explicar:

1. por que $N$ pacotes em $H$ enlaces requerem $N + H - 1$ intervalos de transmissão no modelo ideal;
2. a diferença entre *store-and-forward* e *cut-through*;
3. por que $\mu = R/L$ e $\rho = \lambda L/R$;
4. por que o atraso pode crescer fortemente antes de a utilização atingir 100%;
5. como usar $\overline{N} = \lambda \overline{T}$ sem misturar sistema e fila.

**Para pensar.**

1. Um enlace de 10 Mbit/s recebe 800 pacotes/s de tamanho médio 1,000 bytes. Qual é $\rho$? O sistema pode ser estável?
2. Se $\overline{N} = 12$ pacotes e $\lambda = 600$ pacotes/s, qual é o tempo médio no sistema?
3. Dois enlaces têm a mesma utilização média. Eles necessariamente têm o mesmo atraso médio? Que outras características podem ser relevantes?

**AULA 6 | 28/08/2026 – sexta-feira**

# Teoria de filas

**Cobertura:** Slide 52.

## 6.1 Lei de Little e intensidade de tráfego

Nesta aula, introduzimos um modelo simples para estudar o desempenho de um enlace. O objetivo é relacionar:

- a taxa de chegada de pacotes;
- a capacidade de transmissão do enlace;
- a utilização do enlace;
- o número médio de pacotes;
- o tempo médio de espera;
- o tempo médio total no sistema.

Inicialmente, apresentaremos relações que valem para uma classe ampla de sistemas. Posteriormente, adotaremos as hipóteses específicas do modelo $M/M/1$ para obter fórmulas explícitas para o atraso.

### 6.1.1 Fila de espera, servidor e sistema

Em um instante $t$, definimos:

- $N_q(t)$: número de pacotes esperando na fila;
- $N_s(t)$: número de pacotes no servidor, isto é, sendo transmitidos;
- $N(t)$: número total de pacotes no sistema.

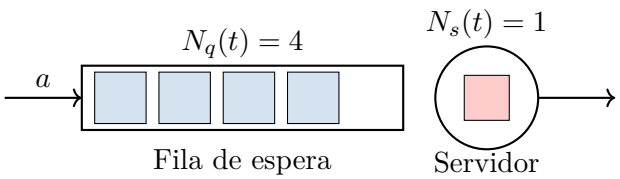
Consequentemente,

$$N(t) = N_q(t) + N_s(t).$$

Em um enlace com um único servidor, temos

$$N_s(t) \in \{0, 1\}.$$

O servidor contém um pacote quando o enlace está ocupado e nenhum pacote quando o enlace está ocioso.




$$N(t) = N_q(t) + N_s(t) = 5$$

É importante distinguir a fila de espera do sistema completo:

$$\boxed{\text{sistema} = \text{fila de espera} + \text{servidor}}.$$

Portanto, “fila de espera” e “sistema” não são sinônimos.

### 6.1.2 Valores instantâneos e valores médios

As variáveis $N_q(t)$, $N_s(t)$ e $N(t)$ descrevem o sistema em um instante. Ao observarmos o sistema durante um intervalo longo, obtemos os valores médios:

- $\overline{N}_q$: número médio de pacotes na fila de espera;
- $\overline{N}_s$: número médio de pacotes no servidor;
- $\overline{N}$: número médio de pacotes no sistema.

Como a igualdade

$$N(t) = N_q(t) + N_s(t)$$

vale em todos os instantes, também temos

$$\boxed{\overline{N} = \overline{N}_q + \overline{N}_s}.$$

Embora o número observado de pacotes seja sempre inteiro, seu valor médio pode ser fracionário. Por exemplo, $\overline{N}_s = 0{,}8$ significa que o servidor está ocupado durante 80% do tempo e vazio durante os outros 20%.

### 6.1.3 Tempos no sistema

Definimos:

- $W$: tempo médio que um pacote passa esperando na fila;
- $S$: tempo médio de serviço;
- $T$: tempo médio total que um pacote passa no sistema.

Consequentemente,

$$\boxed{T = W + S}.$$

Considere um enlace com taxa $R$ bits/s e pacotes com tamanho médio $L$ bits. O tempo médio de serviço é

$$S = \frac{L}{R}.$$

A taxa média de serviço, medida em pacotes por segundo, é

$$\boxed{\mu = \frac{1}{S} = \frac{R}{L}}.$$

Por exemplo, se

$$R = 10\,000\,000 \text{ bits/s}$$

e

$$L = 8\,000 \text{ bits},$$

então

$$\mu = \frac{10\,000\,000}{8\,000} = 1\,250 \text{ pacotes/s}.$$

### 6.1.4 Lei de Little

Seja $a$ a taxa média efetiva de chegada de pacotes, medida em pacotes por segundo.

A Lei de Little afirma que

$$\boxed{\text{número médio} = \text{taxa média de chegada} \times \text{tempo médio}}.$$

Podemos aplicá-la separadamente à fila de espera, ao servidor e ao sistema completo:

$$\overline{N}_q = aW, \tag{27}$$
$$\overline{N}_s = aS, \tag{28}$$
$$\overline{N} = aT. \tag{29}$$

A região usada para contar os pacotes deve ser a mesma região usada para medir o tempo. Por exemplo:

- $\overline{N}_q$ é associado apenas ao tempo de espera $W$;
- $\overline{N}_s$ é associado apenas ao tempo de serviço $S$;
- $\overline{N}$ é associado ao tempo total $T = W + S$.

#### 6.1.4.1 Intuição para a Lei de Little

Considere um intervalo longo de duração $t$. Nesse intervalo, chegam aproximadamente

$$at$$

pacotes.

Se cada pacote passa, em média, $T$ segundos no sistema, esses pacotes acumulam aproximadamente

$$(at)T$$

pacote-segundos.

Por outro lado, se há, em média, $\overline{N}$ pacotes no sistema, a área sob a curva que representa o número de pacotes presentes é aproximadamente

$$\overline{N}t.$$

As duas expressões contam a mesma quantidade. Logo,

$$\overline{N}t = (at)T.$$

Cancelando $t$, obtemos

$$\overline{N} = aT.$$

#### 6.1.4.2 Exemplo

Um enlace recebe, em média,

$$a = 200 \text{ pacotes/s}.$$

Cada pacote passa, em média,

$$T = 15 \text{ ms} = 0{,}015 \text{ s}$$

no sistema. Pela Lei de Little,

$$\overline{N} = aT = 200 \times 0{,}015 = 3.$$

Portanto, há, em média, três pacotes no sistema, contando os pacotes na fila e o pacote em transmissão.

### 6.1.5 Intensidade de tráfego

A intensidade de tráfego é definida por

$$\boxed{I = \frac{aL}{R}}.$$

Como

$$\mu = \frac{R}{L},$$

podemos escrever

$$\boxed{I = \frac{a}{\mu}}.$$

Em teoria de filas, é comum representar a intensidade de tráfego pela letra $\rho$. Assim,

$$\rho = \frac{a}{\mu} \qquad \text{e, nestas notas,} \qquad I = \rho.$$

A quantidade $I$ não possui unidade:

$$I = \frac{\text{pacotes/s}}{\text{pacotes/s}}.$$

### 6.1.6 Intensidade de tráfego e utilização

Seja $U$ a fração do tempo durante a qual o servidor está ocupado.

Quando o servidor está ocupado, ele transmite à taxa $\mu = R/L$ pacotes/s. Quando está vazio, sua taxa de saída é zero. Portanto, a taxa média de saída é

$$U\frac{R}{L} + (1-U)0.$$

Em equilíbrio, a taxa média de saída deve ser igual à taxa média de entrada:

$$a = U\frac{R}{L}.$$

Isolando $U$, obtemos

$$U = \frac{aL}{R}.$$

Como $I = aL/R$,

$$\boxed{U = I = \rho}.$$

Assim, a intensidade de tráfego também representa a fração do tempo durante a qual o enlace está ocupado.

### 6.1.7 Número médio de pacotes no servidor

Como há apenas um servidor,

$$N_s(t) = \begin{cases} 1, & \text{se o servidor estiver ocupado,} \\ 0, & \text{se o servidor estiver vazio.} \end{cases}$$

Consequentemente,

$$\overline{N}_s = 1 \cdot U + 0 \cdot (1-U) \tag{30}$$
$$= U. \tag{31}$$

Como $U = I$,

$$\boxed{\overline{N}_s = I}.$$

Podemos obter o mesmo resultado diretamente pela Lei de Little:

$$\overline{N}_s = aS = a\frac{L}{R} = I.$$

Por exemplo, se $I = 0{,}8$, então

$$\overline{N}_s = 0{,}8.$$

Isso significa que o servidor está ocupado durante 80% do tempo e vazio durante 20% do tempo. Não significa que exista uma fração de pacote no servidor.

### 6.1.8 Condição de estabilidade

Para que o sistema seja estável, a taxa média de chegada deve ser inferior à taxa média de serviço:

$$a < \mu.$$

Equivalentemente,

$$\boxed{I < 1}.$$

Se $I > 1$, chegam mais pacotes por segundo do que o enlace consegue transmitir.

- Com buffer ilimitado, a fila cresce sem limite.
- Com buffer finito, pacotes acabam sendo descartados.
- Mesmo quando $I < 1$, podem ocorrer filas devido às flutuações aleatórias das chegadas.

Uma utilização média inferior a 1 é necessária para a estabilidade, mas não garante que o atraso seja pequeno.

### 6.1.9 O que a Lei de Little não determina

A Lei de Little fornece as relações

$$\overline{N}_q = aW, \qquad \overline{N}_s = aS, \qquad \overline{N} = aT,$$

mas não determina, sozinha, os valores de $W$, $T$ ou $\overline{N}$.

Para obter fórmulas explícitas para essas quantidades, precisamos especificar um modelo para:

- o processo de chegada;
- a distribuição dos tempos de serviço;
- o número de servidores;
- a política de atendimento;
- o tamanho do buffer.

A seguir, adotaremos o modelo $M/M/1$.

## 6.2 A fila $M/M/1$

O modelo $M/M/1$ possui as seguintes hipóteses:

- as chegadas formam um processo de Poisson com taxa $a$;

- os tempos de serviço são exponenciais com média $1/\mu = L/R$;
- há um único servidor;
- o buffer é ilimitado;
- os pacotes são atendidos por ordem de chegada;
- $a < \mu$.

A hipótese exponencial implica a propriedade de falta de memória:

$$\Pr(S > x + y \mid S > x) = \Pr(S > y).$$

Portanto, quando um pacote chega e encontra outro pacote sendo transmitido, o tempo residual médio dessa transmissão continua sendo

$$\frac{1}{\mu}.$$

### 6.2.1 Tempo médio na fila

Um pacote que chega encontra, em média:

- $\overline{N}_q$ pacotes esperando;
- um pacote no servidor com probabilidade $I$.

Como as chegadas são Poisson, uma chegada observa as médias temporais do sistema. Essa propriedade é conhecida como PASTA: *Poisson Arrivals See Time Averages*.

Cada pacote à frente requer, em média, $1/\mu$ segundos de serviço. Assim,

$$W = \left(\overline{N}_q + I\right) \frac{1}{\mu}.$$

Pela Lei de Little,

$$\overline{N}_q = aW.$$

Substituindo,

$$W = (aW + I) \frac{1}{\mu}.$$

Multiplicando por $\mu$,

$$\mu W = aW + I.$$

Portanto,

$$(\mu - a)W = I$$

e

$$W = \frac{I}{\mu - a}.$$

Como

$$I = \frac{a}{\mu},$$

temos

$$\mu - a = \mu(1 - I).$$

Logo,

$$\boxed{W = \frac{I}{\mu(1-I)} = \frac{I\,L/R}{1-I}}.$$

### 6.2.2 Tempo médio total no sistema

O tempo total no sistema é

$$T = W + \frac{1}{\mu}.$$

Substituindo a expressão de $W$,

$$T = \frac{I}{\mu(1-I)} + \frac{1}{\mu} \tag{32}$$

$$= \frac{I}{\mu(1-I)} + \frac{1-I}{\mu(1-I)} \tag{33}$$

$$= \frac{1}{\mu(1-I)}. \tag{34}$$

Portanto,

$$\boxed{T = \frac{1}{\mu(1-I)} = \frac{1}{\mu - a}}.$$

Usando $\mu = R/L$ e $I = aL/R$, também podemos escrever

$$\boxed{T = \frac{L/R}{1 - aL/R}}.$$

### 6.2.3 Número médio de pacotes

Pela Lei de Little,

$$\overline{N} = aT.$$

Substituindo $T = 1/(\mu - a)$,

$$\overline{N} = \frac{a}{\mu - a}.$$

Dividindo o numerador e o denominador por $\mu$,

$$\boxed{\overline{N} = \frac{I}{1 - I}}.$$

O número médio de pacotes na fila de espera é

$$\overline{N}_q = aW.$$

Portanto,

$$\overline{N}_q = a\frac{I}{\mu(1 - I)} \tag{35}$$

$$= \frac{I^2}{1 - I}. \tag{36}$$

Assim,

$$\boxed{\overline{N}_q = \frac{I^2}{1 - I}}.$$

Podemos verificar que

$$\overline{N}_q + \overline{N}_s = \frac{I^2}{1 - I} + I \tag{37}$$

$$= \frac{I^2 + I(1 - I)}{1 - I} \tag{38}$$

$$= \frac{I}{1 - I} \tag{39}$$

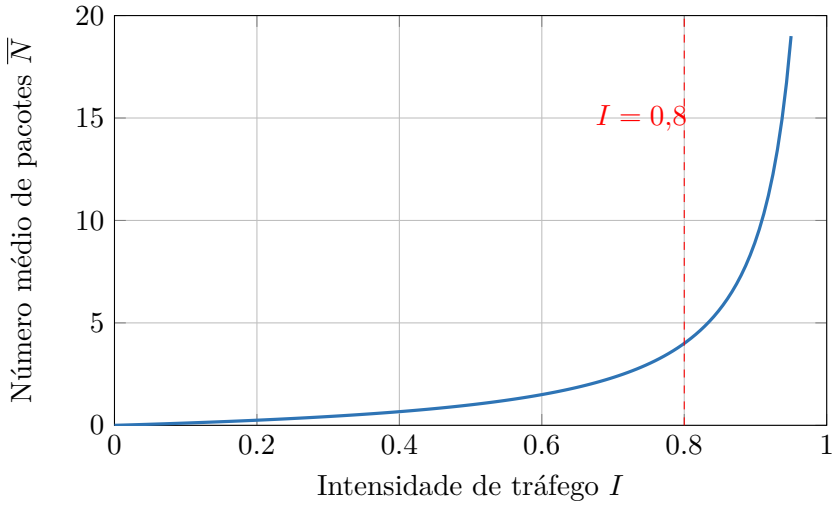
$$= \overline{N}. \tag{40}$$

### 6.2.4 Crescimento do número médio de pacotes

A expressão

$$\overline{N} = \frac{I}{1 - I}$$

mostra que o número médio de pacotes cresce rapidamente quando $I$ se aproxima de 1.




Alguns valores ilustrativos são:

| $I$ | $\overline{N} = I/(1-I)$ |
|---|---|
| 0,20 | 0,25 |
| 0,50 | 1 |
| 0,80 | 4 |
| 0,90 | 9 |
| 0,95 | 19 |

Aumentar a utilização de 80% para 90% mais do que duplica o número médio de pacotes:

$$4 \longrightarrow 9.$$

### 6.2.5 Exemplo numérico

Considere um enlace com

$$R = 10 \text{ Mbit/s}, \qquad L = 1\,000 \text{ bytes}, \qquad a = 800 \text{ pacotes/s}.$$

Convertendo o tamanho dos pacotes para bits,

$$L = 8\,000 \text{ bits}.$$

A taxa de serviço é

$$\mu = \frac{10\,000\,000}{8\,000} = 1\,250 \text{ pacotes/s}.$$

A intensidade de tráfego é

$$I = \frac{a}{\mu} = \frac{800}{1\,250} = 0{,}64.$$

Como $I < 1$, o sistema pode ser estável. No modelo $M/M/1$,

$$\overline{N} = \frac{0{,}64}{1 - 0{,}64} \approx 1{,}78 \text{ pacotes.}$$

O tempo médio total é

$$T = \frac{\overline{N}}{a} = \frac{1{,}78}{800} \approx 0{,}00222 \text{ s.}$$

Portanto,

$$\boxed{T \approx 2{,}22 \text{ ms}}.$$

O tempo médio de serviço é

$$S = \frac{L}{R} = 0{,}8 \text{ ms.}$$

Logo, o tempo médio na fila é

$$W = T - S \approx 2{,}22 - 0{,}8 = 1{,}42 \text{ ms.}$$

### 6.2.6 E se todos os pacotes tiverem o mesmo tamanho?

A fórmula

$$\overline{N} = \frac{I}{1 - I}$$

não vale para qualquer sistema com intensidade $I$. Ela foi obtida usando as hipóteses do modelo $M/M/1$, especialmente a hipótese de tempos de serviço exponenciais.

Se todos os pacotes tiverem exatamente $L$ bits, o tempo de serviço será determinístico. Com chegadas Poisson, o modelo correspondente será $M/D/1$.

Na fila $M/M/1$,

$$W_{M/M/1} = \frac{I}{\mu(1 - I)}.$$

Na fila $M/D/1$,

$$W_{M/D/1} = \frac{I}{2\mu(1 - I)}.$$

Portanto,

$$W_{M/D/1} = \frac{1}{2} W_{M/M/1}.$$

A Lei de Little continua válida nos dois casos. O que muda é o atraso obtido a partir do modelo. A maior variabilidade dos tempos de serviço no modelo $M/M/1$ aumenta o tempo médio de espera.

## 6.3 Síntese

As principais relações desta aula são:

$$\boxed{\mu = \frac{R}{L}}$$

$$\boxed{I = \frac{a}{\mu} = \frac{aL}{R}}$$

$$\boxed{\overline{N}_q = aW, \qquad \overline{N}_s = aS, \qquad \overline{N} = aT}$$

$$\boxed{\overline{N}_s = I}$$

Para o modelo $M/M/1$,

$$\boxed{W = \frac{I}{\mu(1-I)}}$$

$$\boxed{T = \frac{1}{\mu(1-I)} = \frac{1}{\mu - a}}$$

$$\boxed{\overline{N}_q = \frac{I^2}{1-I}, \qquad \overline{N} = \frac{I}{1-I}}.$$

## 6.4 Questões de revisão

1. Qual é a diferença entre fila de espera, servidor e sistema?
2. O que significa afirmar que $\overline{N}_s = 0{,}75$?
3. Por que a intensidade de tráfego também representa a utilização do servidor?
4. Por que $I < 1$ é uma condição necessária para a estabilidade?
5. A Lei de Little depende de chegadas Poisson ou de tempos de serviço exponenciais?
6. A fórmula
$$\overline{N} = \frac{I}{1-I}$$
vale para qualquer fila? Quais hipóteses foram usadas para obtê-la?
7. Dois sistemas com a mesma intensidade de tráfego necessariamente apresentam o mesmo atraso médio?
8. Um enlace de 20 Mbit/s recebe 1 000 pacotes/s, com tamanho médio de 1 500 bytes. Calcule $\mu$ e $I$.
9. Em um sistema, há em média oito pacotes e a taxa de chegada é 400 pacotes/s. Use a Lei de Little para determinar o tempo médio no sistema.
10. Em uma fila $M/M/1$, o que acontece com $W$, $T$ e $\overline{N}$ quando $I$ se aproxima de 1?

**AULA 7 | 02/09/2026 – quarta-feira**

# Modelos probabilísticos, revisão de filas e camadas de protocolo

**Cobertura:** Revisão do Slide 52

## 7.1 Observações sobre o teste

A aula começou com comentários sobre o teste. Quatro pontos foram destacados.

### 7.1.1 Modelagem de situações de incerteza

O primeiro ponto é reconhecer qual distribuição descreve cada situação. Cada uma responde a uma pergunta diferente.

- $Z \sim \text{Ber}(p)$: situações de sucesso ou fracasso, isto é, um único experimento com dois resultados possíveis;
- $X \sim \text{Bin}(n, p)$: situações em que se deseja contar a quantidade de sucessos em $n$ tentativas independentes;
- $Y \sim \text{Geo}(p)$: situações em que se deseja contar a quantidade de tentativas até o primeiro sucesso;
- $W \sim \text{Poi}(\lambda)$: situações em que se deseja contar o número de ocorrências em um dado intervalo de tempo.

A variável binomial é construída a partir de variáveis de Bernoulli:

$$X = \sum_{i=1}^{n} Z_i.$$

A variável geométrica também é construída a partir das mesmas variáveis, mas registrando a posição do primeiro sucesso:

$$Y = \arg\min_i \{Z_i : Z_i = 1\}.$$

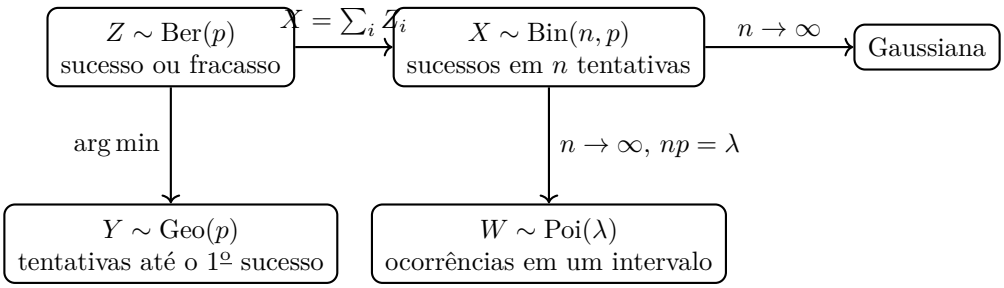
Na variável de Poisson, o parâmetro é o produto da taxa pelo intervalo de interesse:

$$\boxed{\lambda = \tilde{\lambda} T},$$

em que $\tilde{\lambda}$ é a taxa de ocorrências por unidade de tempo e $T$ é a duração do intervalo observado.

#### 7.1.1.1 Relações entre as distribuições

As distribuições não são independentes umas das outras. Há uma conexão entre todas elas:




Dois limites foram destacados em aula:

- quando $n \to \infty$, a soma $X = \sum_i Z_i$ se aproxima de uma Gaussiana;
- quando $n \to \infty$ com $np = \lambda$ fixo, a binomial se aproxima de uma Poisson. Em sala, comentou-se que valores de $n$ a partir de aproximadamente 10 já produzem uma aproximação razoável.

Uma diferença estrutural importante é o suporte de cada variável:

- o suporte da binomial é finito, $X \in \{0, 1, 2, \ldots, n\}$;
- o suporte da geométrica é infinito, $Y \in \{1, 2, 3, \ldots\}$;
- o suporte da Poisson é infinito, $W \in \{0, 1, 2, \ldots\}$.

Por esse motivo, muitas vezes é mais conveniente trabalhar com a Poisson do que com a binomial: a expressão da Poisson depende de um único parâmetro.

**Exemplo de escolha do parâmetro.** Se escolhermos $\lambda = 7$ em uma janela de 5 minutos, isso significa que, na média, chegam 7 clientes a cada 5 minutos. O parâmetro da Poisson é sempre um número médio de ocorrências, não uma probabilidade.

### 7.1.2 PMF aplicada em um ponto

A função massa de probabilidade (PMF) quantifica a probabilidade de uma realização específica da variável aleatória.

Para a binomial,

$$\Pr(X = k) = \binom{n}{k} p^k (1-p)^{n-k}, \qquad k \in \{0, 1, \ldots, n\}.$$

O coeficiente $\binom{n}{k}$ é a combinação de $n$ elementos tomados $k$ a $k$; ele conta de quantas maneiras os $k$ sucessos podem se distribuir entre as $n$ tentativas.

Para a geométrica,

$$\Pr(Y = k) = (1-p)^{k-1} p, \qquad k \in \{1, 2, 3, \ldots\}.$$

Observe que a geométrica começa em 1: é preciso ao menos uma tentativa para haver um primeiro sucesso.

Para a Poisson,

$$\boxed{\Pr(W = w) = \frac{e^{-\lambda T} (\lambda T)^w}{w!}}, \qquad w \in \{0, 1, 2, \ldots\}.$$

### 7.1.3 Notação da CDF

A função de distribuição acumulada (CDF) é definida por

$$\boxed{F_X(k) = \Pr(X \leq k)}.$$

A CDF acumula a PMF até o ponto $k$, inclusive. A distinção entre $\leq$ e $<$ é essencial em variáveis discretas.

### 7.1.4 Cálculo de $\Pr(X \geq k)$

Seja $X \sim \text{Bin}(n, p)$. Há duas formas de calcular $\Pr(X \geq k)$.

**(a) Soma direta de PMFs.**

$$\Pr(X \geq k) = \sum_{i=k}^{n} \binom{n}{i} p^i (1-p)^{n-i}.$$

**(b) Complementar da CDF aplicada em um ponto.**

$$\Pr(X \geq k) = 1 - \Pr(X \leq k-1) = 1 - F_X(k-1).$$

A estratégia é acumular até um valor e depois complementar. O cuidado está no índice: o complementar de $X \geq k$ é $X \leq k-1$, e não $X \leq k$.

**Exemplo discutido em sala.** Arremessos de bola de basquete em uma cesta, com

$$n = 10 \text{ (jogou 10 vezes)}, \qquad p = \text{probabilidade de acertar uma tentativa.}$$

Cada arremesso é uma variável de Bernoulli, e o total de acertos é binomial.

## 7.2 Revisão da aula passada

A segunda parte da aula retomou o modelo de filas visto na Aula 6.

### 7.2.1 Nota sobre notação

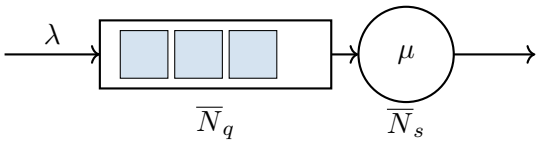
O quadro desta aula usa $\lambda$ para a taxa média de chegada e barras sobre os tempos médios. A correspondência com a notação da Aula 6 é:

$$\lambda = a, \qquad \overline{W} = W, \qquad \overline{S} = S, \qquad \overline{T} = T.$$

Nesta aula, adotamos a notação do quadro.

### 7.2.2 Os três elementos

O sistema estudado tem três elementos: a taxa de chegada, a fila e o servidor.




O tempo de serviço médio é

$$\boxed{\overline{S} = \frac{1}{\mu} = \frac{\overline{L}}{R}},$$

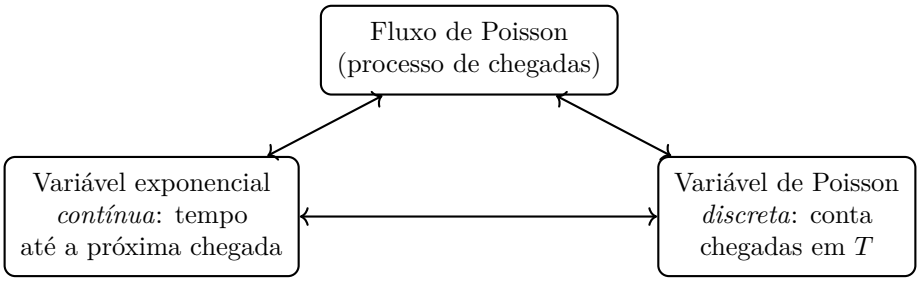
em que $\overline{L}$ é o tamanho médio do pacote e $R$ é a taxa do enlace.

A intensidade de tráfego é a probabilidade de o servidor estar ocupado:

$$I = \Pr(\text{ocupado}) < 1.$$

### 7.2.3 Fluxo de Poisson, variável de Poisson e variável exponencial

Um ponto central da revisão é que "Poisson" aparece em dois papéis diferentes: como fluxo de chegadas e como variável aleatória discreta. Ligada a esse fluxo há ainda uma variável contínua, a exponencial.




A variável de Poisson responde à pergunta: qual a probabilidade de haver $w$ chegadas em um intervalo $T$ qualquer?

$$\Pr(W = w) = \frac{e^{-\lambda T}(\lambda T)^w}{w!}.$$

A variável exponencial responde à pergunta complementar. Definindo

$$V \triangleq \text{tempo até a próxima chegada},$$

o evento $V > T$ significa que nenhuma chegada ocorreu no intervalo $T$. Logo,

$$\boxed{\Pr(V > T) = \Pr(W = 0) = e^{-\lambda T}}.$$

Essa é a ponte entre as duas descrições: contar chegadas em um intervalo (discreto) e medir o tempo entre chegadas (contínuo) são duas leituras do mesmo fluxo.

### 7.2.4 Chegadas de Poisson e o estado do sistema

Como as chegadas são de Poisson, um pacote que chega enxerga o sistema no seu estado médio de longo prazo. Ele não chega em um instante privilegiado. Essa é a propriedade PASTA, *Poisson Arrivals See Time Averages*, apresentada na Aula 6.

### 7.2.5 A conta do tempo médio de espera

Um pacote que chega precisa esperar que todo o trabalho pendente à sua frente seja executado. Esse trabalho pendente é composto por:

- os $\overline{N}_q$ pacotes que estão na fila;
- o pacote que está no servidor, presente em média $\overline{N}_s = I$.

Cada um deles consome, em média, $\overline{S}$ segundos. Portanto,

$$\boxed{\overline{W} = \left(\overline{N}_q + \overline{N}_s\right)\overline{S}}.$$

Duas hipóteses sustentam essa expressão:

1. **PASTA**: a chegada observa as médias temporais do sistema;
2. **falta de memória do serviço**: o pacote que já está sendo transmitido ainda demanda, em média, $\overline{S}$ segundos, e não menos.

Aplicando a Lei de Little, $\overline{N}_q = \lambda\overline{W}$, e usando $\overline{N}_s = I$:

$$\overline{W} = \lambda\overline{W}\,\overline{S} + I\,\overline{S}. \tag{41}$$

Como $\lambda\overline{S} = \lambda/\mu = I$,

$$\overline{W} = I\,\overline{W} + I\,\overline{S}.$$

Isolando $\overline{W}$,

$$\overline{W}(1 - I) = I\,\overline{S}$$

e portanto

$$\boxed{\overline{W} = \frac{I\,\overline{S}}{1 - I}}.$$

### 7.2.6 Número médio de pacotes

Pela Lei de Little aplicada à fila de espera,

$$\overline{N}_q = \lambda\overline{W} = \lambda\frac{I\,\overline{S}}{1 - I} = \frac{I^2}{1 - I},$$

pois $\lambda\overline{S} = I$. Assim,

$$\boxed{\overline{N}_q = \frac{I^2}{1-I}}.$$

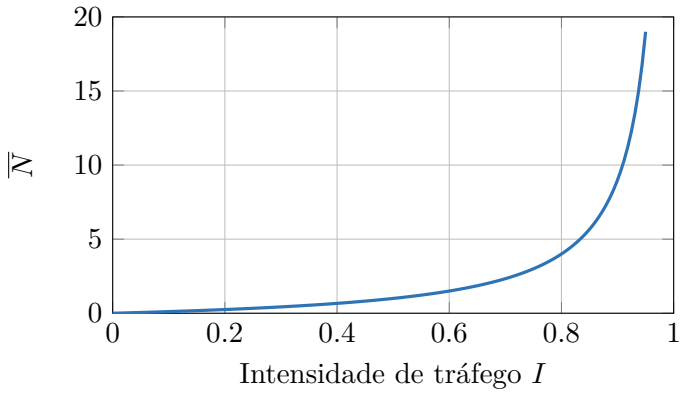
Somando o pacote em serviço,

$$\overline{N} = \overline{N}_q + \overline{N}_s = \frac{I^2}{1-I} + I = \frac{I}{1-I}.$$

Portanto,

$$\boxed{\overline{N} = \frac{I}{1-I}}.$$

O gráfico de $\overline{N}$ contra $I$, revisto em aula, cresce sem limite quando $I \to 1$:




### 7.2.7 Condição de equilíbrio

Foi retomada a observação do Kurose: o sistema só está em equilíbrio se a taxa de chegada for menor do que a capacidade de serviço. Essa é uma condição *necessária* para o sistema estar em equilíbrio, e corresponde a $I < 1$.

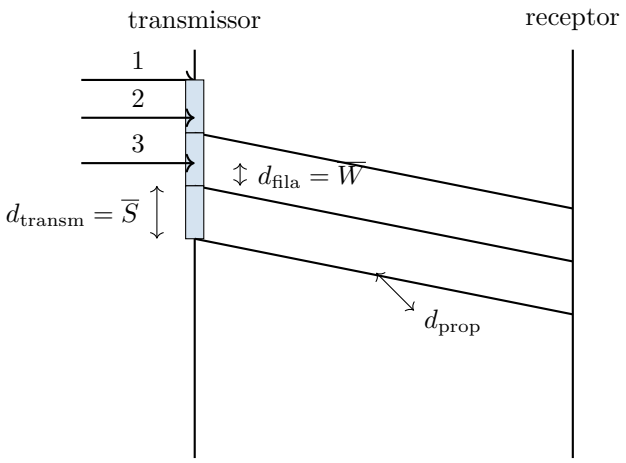
Além disso, se $\overline{S}$ aumenta, então $\overline{W}$ e $\overline{T}$ aumentam. Serviços mais lentos elevam a intensidade de tráfego e, com ela, o atraso.

## 7.3 Decomposição do atraso

O tempo total $\overline{T}$ percebido por um pacote se decompõe em parcelas distintas:

- $d_{\text{fila}} = \overline{W}$: espera na fila;
- $d_{\text{transm}} = \overline{S}$: tempo de transmissão, isto é, de serviço;
- $d_{\text{prop}}$: tempo de propagação no enlace.

O diagrama espaço-tempo torna a distinção visível. O eixo vertical representa o tempo, e as duas linhas verticais representam o transmissor e o receptor.

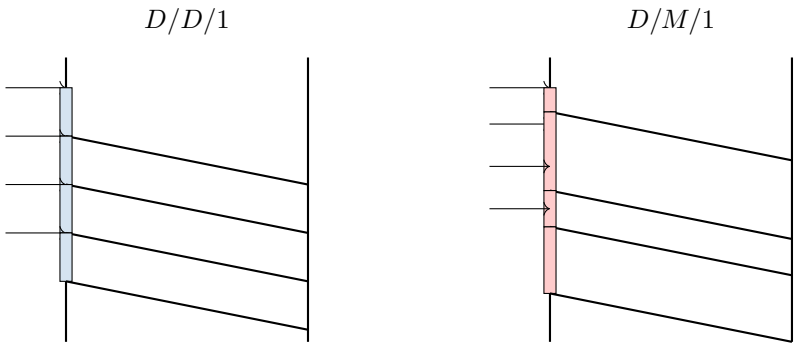



A altura do retângulo de cada pacote é o tempo de transmissão; a inclinação da reta é o tempo de propagação. A fila aparece como o intervalo entre a chegada do pacote e o início da sua transmissão.

## 7.4 Efeito da variabilidade: $M/D/1$ e $M/M/1$

Na fila $M/D/1$, o tempo de serviço é determinístico: todos os pacotes têm o mesmo tamanho, então todos os retângulos de transmissão têm a mesma altura e as retas de propagação são paralelas.

Na fila $M/M/1$, o tempo de serviço é exponencial: o tamanho do pacote é uma variável aleatória, então os retângulos têm alturas diferentes.




A comparação quantitativa foi vista na Aula 6:

$$\overline{W}_{M/D/1} = \frac{1}{2}\,\overline{W}_{M/M/1}.$$

A conclusão é que a variabilidade dos tempos de serviço, e não apenas a média, determina o atraso. Dois sistemas com a mesma intensidade de tráfego $I$ podem ter atrasos médios diferentes.

## 7.5 Roteiro do Capítulo 1

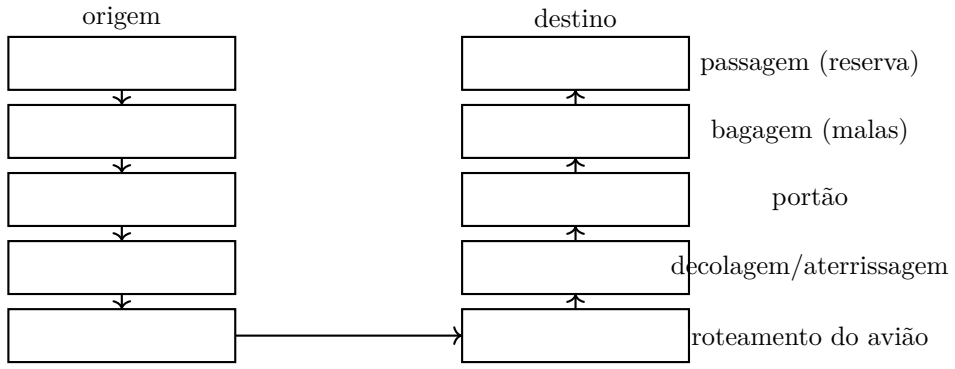
- o que é a Internet;
- a borda da rede (*edge*);
- o núcleo da rede (*core*);
- atrasos e perdas;
- modelo em camadas;
- segurança;
- histórico.

O item de segurança veio acompanhado da referência ao livro *Fancy Bear Goes Phishing*, indicado como leitura complementar.

## 7.6 Camadas de protocolo

### 7.6.1 A analogia da companhia aérea

O Kurose apresenta a arquitetura em camadas por meio de uma analogia humana: o sistema de uma companhia aérea. Esse sistema envolve emissão de passagens, despacho de bagagem, portão de embarque, pilotos, aviões e controle de tráfego aéreo. Organizar essas funções em camadas permite descrever o sistema por partes.




Cada camada, combinada com as que estão abaixo dela, implementa um serviço:

- na camada da passagem e abaixo dela, realiza-se a transferência balcão-a-balcão do passageiro;
- na camada da bagagem e abaixo dela, realiza-se a transferência despacho-a-recuperação do passageiro e de suas malas;
- na camada do portão, realiza-se a transferência portão-a-portão;
- na camada de decolagem/aterrissagem, realiza-se a transferência pista-a-pista.

Cada camada provê seu serviço de duas maneiras: executando ações próprias e utilizando os serviços da camada imediatamente inferior.

Assim como as funções da companhia aérea estão distribuídas entre vários aeroportos, um protocolo de camada $n$ está distribuído entre sistemas finais, comutadores de pacote e demais componentes da rede.

### 7.6.2 Vantagens e desvantagens

A principal vantagem é a modularidade: a divisão em camadas permite discutir uma parcela bem definida de um sistema grande e facilita a atualização de componentes.

- uma camada pode duplicar a funcionalidade de uma camada inferior. Por exemplo, várias pilhas oferecem recuperação de erros por enlace e também fim a fim, o que gera trabalho desnecessário;
- uma camada pode precisar de informação que só existe em outra camada, o que infringe o objetivo de separação entre elas.

O conjunto dos protocolos das várias camadas é denominado *pilha de protocolos*. A pilha da Internet tem cinco camadas: física, de enlace, de rede, de transporte e de aplicação. O livro é organizado nessa ordem, de cima para baixo, o que caracteriza a abordagem *top-down*.

## 7.7 Síntese

Distribuições e suas relações:

$$\boxed{X = \sum_{i=1}^{n} Z_i, \qquad Y = \arg\min_i \{Z_i : Z_i = 1\}, \qquad \lambda = \tilde{\lambda}\, T}$$

$$\boxed{\Pr(X = k) = \binom{n}{k} p^k (1-p)^{n-k}}$$

$$\boxed{\Pr(Y = k) = (1-p)^{k-1} p}$$

$$\boxed{\Pr(W = w) = \frac{e^{-\lambda T}(\lambda T)^w}{w!}}$$

$$\boxed{F_X(k) = \Pr(X \leq k), \qquad \Pr(X \geq k) = 1 - F_X(k-1)}$$

Fluxo de Poisson e tempo entre chegadas:

$$\boxed{\Pr(V > T) = \Pr(W = 0) = e^{-\lambda T}}$$

Filas:

$$\boxed{\overline{S} = \frac{1}{\mu} = \frac{\overline{L}}{R}, \qquad I = \Pr(\text{ocupado}) < 1}$$

$$\boxed{\overline{W} = \left(\overline{N}_q + \overline{N}_s\right) \overline{S} = \frac{I\,\overline{S}}{1-I}}$$

$$\boxed{\overline{N}_q = \frac{I^2}{1-I}, \qquad \overline{N} = \frac{I}{1-I}}$$

Atraso:

$$\boxed{\overline{T} = d_{\text{fila}} + d_{\text{transm}} + d_{\text{prop}}}$$

## 7.8 Questões de revisão

1. Qual pergunta cada uma das distribuições responde: Bernoulli, binomial, geométrica e Poisson?
2. Por que a geométrica tem suporte $\{1, 2, 3, \dots\}$ e não $\{0, 1, 2, \dots\}$?
3. Sob quais condições a binomial se aproxima de uma Poisson? E de uma Gaussiana?
4. Em $\lambda = \tilde{\lambda}\, T$, o que representa cada fator? O que significa afirmar que $\lambda = 7$?
5. Escreva $\Pr(X \geq 3)$ para $X \sim \text{Bin}(10, p)$ das duas formas vistas em aula. Por que o complementar usa $\Pr(X \leq 2)$?
6. Qual é a diferença entre o fluxo de Poisson, a variável de Poisson e a variável exponencial?
7. Deduza $\Pr(V > T) = e^{-\lambda T}$ a partir da PMF de Poisson.
8. Quais são as duas hipóteses usadas para escrever $\overline{W} = (\overline{N}_q + \overline{N}_s)\overline{S}$?
9. Partindo dessa expressão e da Lei de Little, obtenha $\overline{W} = I\overline{S}/(1-I)$.
10. Quais parcelas compõem $\overline{T}$? Qual delas não depende da intensidade de tráfego?
11. Por que $M/D/1$ apresenta metade da espera de $M/M/1$, mesmo com a mesma intensidade de tráfego?
12. Na analogia da companhia aérea, qual serviço é prestado pela camada do portão e abaixo dela?
13. Quais são as cinco camadas da pilha de protocolos da Internet?

**AULA 8 | 04/09/2026 – sexta-feira**

# Tempo residual de serviço, vazão e comunicação entre processos

**Cobertura:** Fim do Capítulo 1 e início do Capítulo 2.

## 8.1 Estabilidade e ocupação do servidor

Há uma distinção que precisa ficar clara: um sistema pode estar com o servidor permanentemente ocupado sem que isso implique instabilidade.

Na fila $M/M/1$, a condição de estabilidade é

$$I < 1,$$

e o sistema é instável caso contrário. Isso decorre das flutuações aleatórias: com chegadas de Poisson e serviço exponencial, mesmo com $I$ próximo de 1 ocorrem rajadas que fazem a fila crescer, e é preciso folga para que ela se esvazie.

### 8.1.1 Existe sistema estável com $I = 1$?

Sim. Considere a fila $D/D/1$:

- chegadas determinísticas;
- tempos de serviço determinísticos;
- um único servidor.

Se um pacote chega exatamente a cada $1/\lambda$ segundos e cada pacote leva exatamente $\overline{S} = 1/\mu$ segundos para ser transmitido, com $\lambda = \mu$, então cada pacote encontra o servidor livre no instante exato em que chega. O servidor fica ocupado 100% do tempo, ou seja, $I = 1$, e mesmo assim a fila nunca cresce: não há espera.

A conclusão é que a condição $I < 1$ é necessária apenas quando existe aleatoriedade. O que gera fila não é a utilização alta em si, mas a variabilidade em torno dela.

## 8.2 Tempo residual de serviço

Um pacote que chega e encontra o servidor ocupado não espera um tempo de serviço completo pelo pacote em transmissão: espera apenas o que falta dele. Essa quantidade é o tempo residual de serviço, denotado $\overline{S}_R$.

### 8.2.1 A expressão geral da espera

A espera se decompõe em duas parcelas:

$$\boxed{\overline{W} = \overline{N}_q\,\overline{S} + \overline{N}_s\,\overline{S}_R}.$$

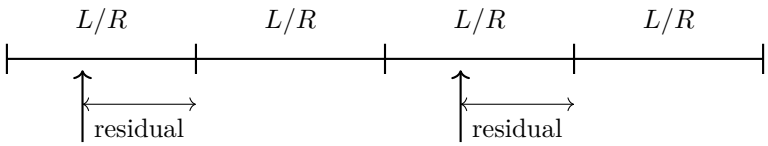
O primeiro termo considera os pacotes que estão inteiramente na fila: cada um deles ainda será transmitido por completo, consumindo $\overline{S}$ segundos. O segundo termo trata do pacote que já está no servidor, e por isso usa o tempo residual e não o tempo completo.

O peso do segundo termo é a probabilidade de haver alguém em serviço:

$$\boxed{\overline{N}_s = I = U = \Pr(\text{ocupação})}.$$

### 8.2.2 Cálculo do tempo residual

Considere o eixo do tempo dividido pelas transmissões sucessivas, cada uma com duração $L/R$. Um pacote que chega cai em um ponto qualquer de um desses intervalos, e o que ele espera é a distância desse ponto até o fim do intervalo.




Como as chegadas não escolhem o instante em que ocorrem, intervalos longos têm mais chance de receber uma chegada do que intervalos curtos, e exatamente na proporção da sua duração. Isso faz o segundo momento aparecer:

$$\boxed{\overline{S}_R = \frac{\overline{S^2}}{2\overline{S}} = \frac{\mathbb{E}(S^2)}{2\,\mathbb{E}(S)}}.$$

O numerador é o segundo momento do tempo de serviço, isto é, a esperança de $S^2$.

### 8.2.3 Os dois casos particulares

#### 8.2.3.1 Serviço exponencial: $M/M/1$

Para o tempo de serviço exponencial de média $1/\mu$,

$$\mathbb{E}(S) = \frac{1}{\mu}, \qquad \mathbb{E}(S^2) = \frac{2}{\mu^2}.$$

Logo,

$$\overline{S}_R = \frac{2/\mu^2}{2/\mu} = \frac{1}{\mu} = \overline{S}.$$

O tempo residual é igual ao tempo de serviço completo. Esse é o significado concreto da propriedade *memoryless*: saber que a transmissão já começou não adianta nada. A espera fica

$$\boxed{\overline{W}_{M/M/1} = \overline{N}_q\,\overline{S} + \overline{N}_s\,\overline{S}}.$$

#### 8.2.3.2 Serviço determinístico: $M/D/1$

Para o tempo de serviço constante e igual a $\overline{S}$,

$$\mathbb{E}(S) = \overline{S}, \qquad \mathbb{E}(S^2) = \overline{S}^{\,2}.$$

Logo,

$$\overline{S}_R = \frac{\overline{S}^{\,2}}{2\overline{S}} = \frac{\overline{S}}{2}.$$

Uma chegada em um instante qualquer encontra, em média, metade da transmissão já concluída. A espera fica

$$\boxed{\overline{W}_{M/D/1} = \overline{N}_q\,\overline{S} + \overline{N}_s\,\frac{\overline{S}}{2}}.$$

#### 8.2.3.3 Resolvendo o caso determinístico

Aplicando a Lei de Little, $\overline{N}_q = \lambda\overline{W}$, e usando $\overline{N}_s = I$ com $\lambda\overline{S} = I$:

$$\overline{W} = \lambda\overline{W}\,\overline{S} + I\,\frac{\overline{S}}{2} \tag{42}$$

$$= I\,\overline{W} + \frac{I\,\overline{S}}{2}. \tag{43}$$

Isolando,

$$\overline{W}(1 - I) = \frac{I\,\overline{S}}{2}$$

e portanto

$$\boxed{\overline{W}_{M/D/1} = \frac{I\,\overline{S}}{2(1-I)} = \frac{1}{2}\,\overline{W}_{M/M/1}}.$$

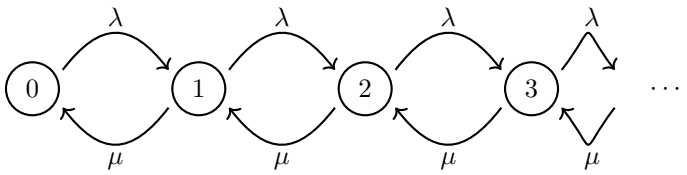
### 8.2.4 O papel da variabilidade

A expressão de $\overline{S}_R$ depende do segundo momento, e não apenas da média. Duas filas podem ter o mesmo $\overline{S}$, a mesma taxa de chegada e a mesma intensidade de tráfego, e ainda assim apresentar esperas diferentes. Quanto maior a variabilidade dos tempos de serviço, maior o tempo residual e pior o desempenho.

Vale notar que várias aplicações geram pacotes com tamanho determinístico: a aplicação usa sempre o mesmo tamanho, o que aproxima o comportamento do caso $M/D/1$ e reduz a espera pela metade em relação ao caso exponencial.

## 8.3 Distribuição do número de pacotes

O sistema pode ser descrito pelos estados $0, 1, 2, 3, \ldots$, em que o estado $n$ significa haver $n$ pacotes no sistema. As transições ocorrem para o estado seguinte à taxa $\lambda$, quando chega um pacote, e para o estado anterior à taxa $\mu$, quando um pacote termina de ser servido.




A probabilidade de o sistema estar vazio é o complementar da probabilidade de estar ocupado:

$$\boxed{\pi_0 = 1 - \Pr(\text{ocupado}) = 1 - \rho = 1 - U = 1 - I}.$$

## 8.4 Vazão

### 8.4.1 Vazão instantânea e vazão média

A vazão instantânea é a taxa com que os bits estão sendo entregues em um dado momento. Enquanto o servidor transmite, ela vale

$$\frac{R}{L} = \mu,$$

e enquanto o servidor está ocioso, ela vale zero.

A vazão média é obtida condicionando nos dois estados possíveis:

$$\boxed{\text{vazão média} = (\text{vazão} \mid \text{ocupado})\Pr(\text{ocupado}) + (\text{vazão} \mid \text{ocioso})\Pr(\text{ocioso})}.$$

Substituindo cada termo,

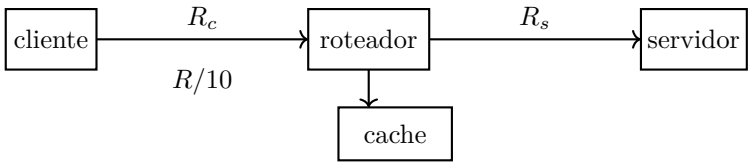
$$\lambda = \mu\, I + 0 \cdot (1 - I).$$

Essa identidade confirma que $I = \lambda/\mu$: em equilíbrio, o que entra é igual ao que sai, e a única forma de o servidor entregar $\lambda$ pacotes por segundo transmitindo à taxa $\mu$ é ficar ocupado durante a fração $I$ do tempo.

### 8.4.2 Vazão fim a fim e gargalo

Em um caminho fim a fim, a vazão é limitada pelo enlace mais restritivo. Considere um servidor com taxa de acesso $R_s$, um cliente com taxa de acesso $R_c$ e um enlace intermediário de taxa $R$ compartilhado por 10 conexões simultâneas. A vazão de cada conexão é

$$\boxed{\min\left(R_c,\ R_s,\ \frac{R}{10}\right)}.$$




Um cache instalado próximo ao cliente altera essa conta. Uma fração $p_{\text{hit}}$ das requisições é atendida localmente, sem atravessar o gargalo; apenas a fração $p_{\text{miss}}$ percorre o caminho completo. O tráfego que efetivamente disputa o enlace compartilhado é reduzido pelo fator $p_{\text{miss}}$, o que diminui a intensidade de tráfego naquele enlace e, com ela, o atraso de fila. O cache do navegador é um exemplo desse mecanismo.

## 8.5 Medição do atraso

### 8.5.1 TTL

Todo pacote carrega um campo TTL, *time to live*, que é decrementado a cada roteador atravessado. Quando o campo chega a zero, o roteador descarta o pacote e devolve uma mensagem de erro à origem. O objetivo do campo é impedir que pacotes circulem indefinidamente em caso de laço de roteamento.

### 8.5.2 Traceroute

O traceroute explora esse mecanismo: envia pacotes com valores crescentes de TTL, de modo que cada roteador do caminho seja obrigado a responder na sua vez. Para cada salto são enviados três *probes*, e os três tempos são reportados.

Um valor medido, como 121 ms, é um tempo total: ele engloba o atraso de fila acumulado no caminho, além dos atrasos de transmissão, propagação e processamento. Por isso os três valores de um mesmo salto costumam diferir entre si.

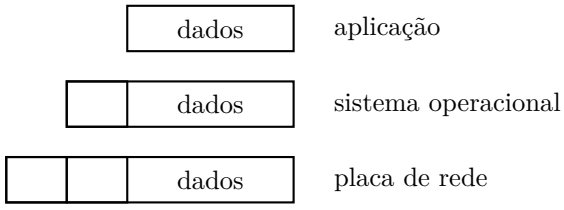
Vale lembrar que $\overline{W}$ é uma média fixa, determinada pelos parâmetros do sistema, enquanto as amostras individuais de espera variam de pacote para pacote.

## 8.6 Custo da divisão em camadas

A modularidade das camadas tem um preço, e ele aparece em dois lugares.

Primeiro, cada camada mantém sua própria fila, o que significa uma fila por camada ao longo do percurso.

Segundo, cada camada atravessada acrescenta um cabeçalho ao pacote. O cabeçalho mais interno é o que só será examinado pelo destino final; os cabeçalhos externos são lidos e removidos pelos elementos intermediários.



## 8.7 Segurança

Três propriedades organizam a discussão de segurança, resumidas pela sigla AIC.

| **Availability** (disponibilidade) | **Integrity** (integridade) | **Confidentiality** (confidencialidade) |
|---|---|---|
| DoS | *spoofing* | *sniffing* |
| ataque ativo | ataque ativo | ataque passivo |
| *firewall* | assinaturas, *hashes* | criptografia |

A distinção entre ataque ativo e passivo é relevante: no ataque ativo há modificação ou injeção de tráfego, o que o torna detectável em princípio; no ataque passivo há apenas observação, e por isso a defesa é impedir a leitura, não a interceptação.

Como leitura complementar sobre o tema, foi indicado o livro *Fancy Bear Goes Phishing*.

## 8.8 Capítulo 2: comunicação entre processos

### 8.8.1 Formas de comunicação

Uma aplicação de rede não é um programa executando em uma máquina isolada: são processos em máquinas diferentes que trocam informação. Antes de tratar da rede, convém situar as formas gerais de comunicação entre processos (IPC):

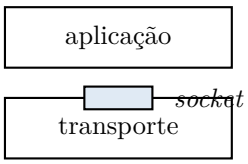
- *pipes*;
- sinais;
- mensagens (*sockets*);
- memória compartilhada.

Dessas quatro, a comunicação por mensagens é a que interessa às redes, pois é a única que funciona entre máquinas distintas. A memória compartilhada pertence ao domínio da computação concorrente, dentro de uma mesma máquina.

Vale a distinção: *threads* já compartilham memória por padrão; processos não.

### 8.8.2 Sockets

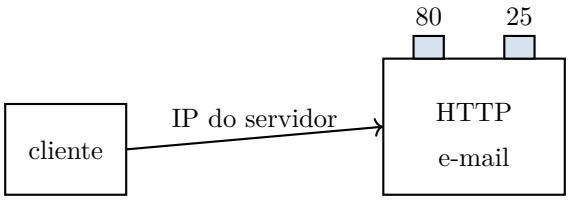
O socket é o elemento entre a aplicação e a camada de transporte. Ele é implementado pelo sistema operacional, e é, concretamente, uma região de memória do sistema operacional.




As operações disponíveis sobre um socket são criar, ler e destruir. A aplicação escreve no socket e lê do socket; tudo o que acontece abaixo dele é responsabilidade do sistema operacional e da rede.

### 8.8.3 Portas

O endereço IP identifica a máquina; a porta identifica qual processo dentro dela deve receber os dados. Um mesmo servidor pode atender simultaneamente à porta 80, para HTTP, e à porta 25, para correio eletrônico.




### 8.8.4 Transporte: TCP e UDP

Há duas opções de transporte disponíveis para a aplicação:

- TCP: oferece muitos serviços à aplicação, e por isso é *heavyweight*;
- UDP: oferece quase nada, é minimalista e *lightweight*.

A escolha entre eles é uma decisão de projeto da aplicação: quanto do trabalho ela quer delegar ao transporte e quanto quer implementar por conta própria.

O HTTP tradicionalmente usa TCP, mas também pode operar sobre QUIC, que por sua vez roda sobre UDP.

O capítulo se encerra com a programação de uma aplicação usando sockets.

## 8.9 Síntese

Espera com tempo residual:

$$\boxed{\overline{W} = \overline{N}_q\,\overline{S} + \overline{N}_s\,\overline{S}_R, \qquad \overline{N}_s = I = U = \Pr(\text{ocupação})}$$

$$\boxed{\overline{S}_R = \frac{\overline{S^2}}{2\overline{S}} = \frac{\mathbb{E}(S^2)}{2\,\mathbb{E}(S)}}$$

$$\boxed{\overline{S}_R^{M/M/1} = \overline{S}, \qquad \overline{S}_R^{M/D/1} = \frac{\overline{S}}{2}}$$

$$\boxed{\overline{W}_{M/D/1} = \frac{I\,\overline{S}}{2(1-I)} = \frac{1}{2}\,\overline{W}_{M/M/1}}$$

Ocupação e vazão:

$$\boxed{\pi_0 = 1 - \rho = 1 - U = 1 - I}$$

$$\boxed{\lambda = \mu\,I + 0 \cdot (1 - I)}$$

$$\boxed{\text{vazão fim a fim} = \min\left(R_c,\ R_s,\ \frac{R}{10}\right)}$$

**AULA 9 | 09/09/2026 – quarta-feira**

# Camada de Aplicação: HTTP, Desempenho e Conexões

**Cobertura:** Resolução de exercícios (Lei de Little), características do protocolo HTTP, modelos de conexão TCP (triângulo de *handshake*, objetos com linha grossa), análise de tempo (RTT) e evolução das versões do HTTP com *interleaving*.

## 9.1 Nota sobre a Lei de Little (Lista de Exercícios)

A partir da discussão do exercício P16 em sala, reforçamos uma premissa fundamental da teoria de filas:

**Cuidado:** Para aplicar a Lei de Little ($\overline{N} = \lambda \overline{T}$), o sistema precisa **necessariamente estar em equilíbrio** ($I < 1$). A questão discutida estava mal formulada porque violava esse princípio. Possíveis consertos matemáticos para tornar o sistema viável envolvem diminuir a taxa de chegada ($\lambda$), aumentar a taxa de serviço ($\mu$) ou fixar $\overline{N}$ e $\overline{T}$ ajustando a entrada.

## 9.2 O Protocolo HTTP

O HTTP (*Hypertext Transfer Protocol*) é o protocolo base da Web, operando na camada de aplicação.

**Protocolo Stateless:** O HTTP é um protocolo sem estado. O conteúdo da resposta não depende do histórico de requisições anteriores. Para manter sessões (como logins), a web utiliza **cookies**.

## 9.3 Modelos de Conexão HTTP

O desempenho do HTTP depende de como ele utiliza as conexões da camada de transporte (TCP). Para iniciar qualquer requisição HTTP, primeiro é necessário estabelecer a conexão TCP através de um *Three-way Handshake*. Isso consome 1 RTT (*Round-Trip Time*) inicial apenas para sincronizar (SYN e SYN/ACK) antes mesmo do GET ser enviado.

A tabela abaixo unifica e resume as quatro abordagens formadas pela combinação da persistência e do paralelismo, oferecendo uma visão global:

| | **Sem Paralelismo** | **Com Paralelismo** |
|---|---|---|
| **Não Persistente** | Fecha conexão após cada objeto; requisições e *handshakes* estritamente sequenciais. | Múltiplas conexões TCP (com seus respectivos *handshakes*) abertas simultaneamente. |
| **Persistente** | Uma única conexão TCP mantida aberta; requisições sequenciais na mesma via. | Reaproveita conexões TCP abertas e abre novas conexões em paralelo quando necessário. |

Abaixo, detalhamos cada modelo com suas vantagens, desvantagens, diagrama de espaço-tempo e análise numérica do tempo envolvido. Nos diagramas, as setas cinzas formam o **triângulo do handshake TCP**, e as setas **mais grossas representam a transferência do objeto**.

### 9.3.1 1. HTTP Não Persistente (Sem Paralelismo)

Neste modelo, uma nova conexão TCP é aberta para solicitar um objeto e imediatamente fechada após a sua recepção. Os objetos são baixados um por vez.

- **Vantagens:** Consome menos recursos ociosos do servidor. Como a porta TCP é liberada rapidamente após o uso, é um modelo mais robusto contra ataques de negação de serviço que exploram conexões presas.
- **Desvantagens:** Pior desempenho de tempo. Paga-se a latência de um *handshake* TCP (1 RTT) para **cada** nova requisição de objeto na página, dobrando o tempo necessário para objetos pequenos.

**Análise de Tempo:** Para baixar o arquivo HTML base e $n$ objetos referenciados (desprezando o tempo de transmissão dos objetos pequenos), gasta-se 2 RTT por arquivo. Tempo total:

$$\text{Tempo Total} \approx 2 \text{ RTT} + 2n \text{ RTT} = 2(n+1) \text{ RTT}$$

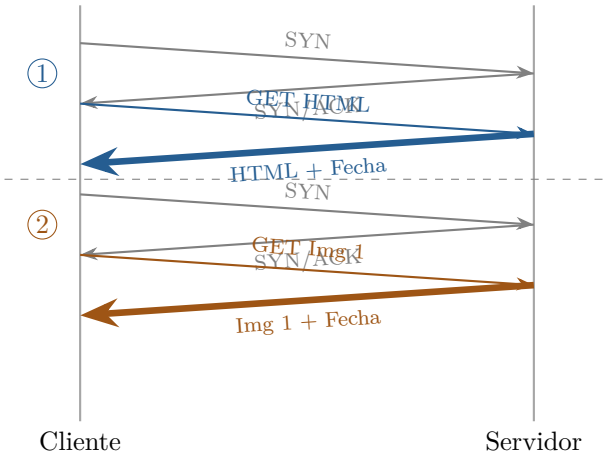



Figure 8: Não Persistente Sem Paralelismo: O "triângulo" do TCP é formado antes de cada GET.

### 9.3.2 2. HTTP Não Persistente (Com Paralelismo)

O cliente abre *múltiplas* conexões TCP simultâneas para baixar os objetos paralelos (ex: imagens) ao mesmo tempo.

- **Vantagens:** O tempo de carregamento da página cai drasticamente, pois os RTTs dos objetos subsequentes são sobrepostos no tempo.
- **Desvantagens:** Exige um consumo severo de recursos. O servidor precisa alocar inúmeros *sockets*, *buffers* e processar vários *handshakes* simultaneamente para um único cliente, sobrecarregando o SO.

**Análise de Tempo:** Gasta-se 2 RTT para o HTML base e, assumindo paralelismo perfeito para os $n$ objetos, mais 2 RTT simultâneos. Tempo total:

$$\text{Tempo Total} \approx 2 \text{ RTT (HTML)} + 2 \text{ RTT (Objetos)} = 4 \text{ RTT}$$

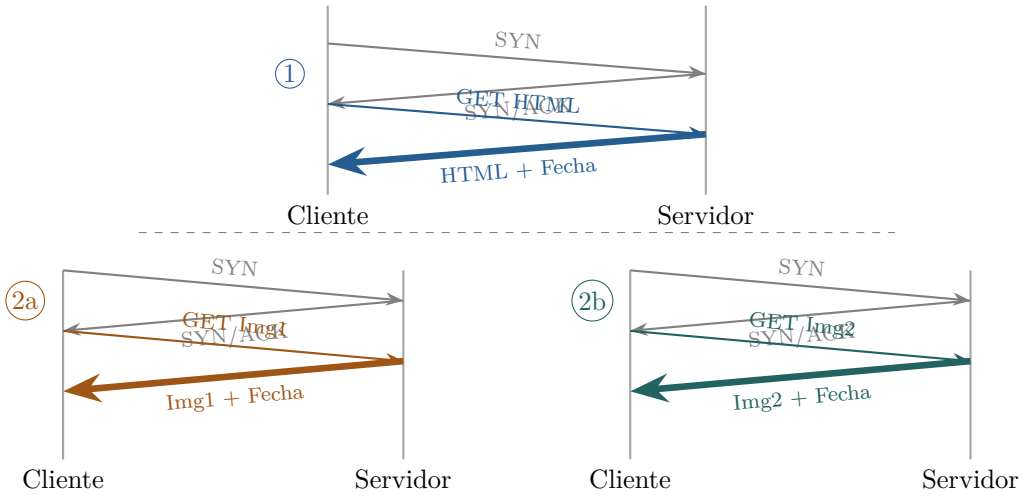



Figure 9: Não Persistente Com Paralelismo: Múltiplas conexões formam seus próprios triângulos e baixam dados ao mesmo tempo.

### 9.3.3 3. HTTP Persistente (Sem Paralelismo)

Apenas um *handshake* TCP é feito. A mesma conexão é mantida aberta. O cliente envia a requisição, aguarda receber o objeto inteiro, e então usa a mesma via para a próxima requisição.

- **Vantagens:** Evita a sobrecarga de abrir novas conexões (economiza 1 RTT de *handshake* por objeto subsequente) e poupa CPU/memória do servidor.
- **Desvantagens:** Mantém a conexão ociosa ocupando recursos do servidor no longo prazo. Continua sofrendo atrasos caso a resposta de um arquivo seja lenta, travando o envio do GET seguinte.

**Análise de Tempo:** 2 RTT para o HTML base (inclui TCP setup) e 1 RTT para cada um dos $n$ objetos adicionais. Tempo total:

$$\text{Tempo Total} \approx 2 \text{ RTT} + n \text{ RTT} = (n + 2) \text{ RTT}$$

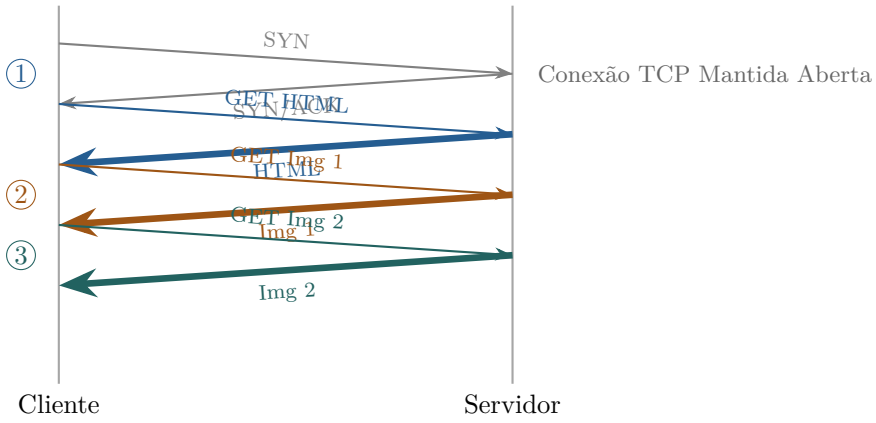



Figure 10: Persistente Sem Paralelismo: Apenas um *handshake* TCP inicial (triângulo superior).

### 9.3.4 4. HTTP Persistente (Com Paralelismo)

Para baixar múltiplos objetos ao mesmo tempo na web moderna, o cliente **reaproveita a conexão persistente já existente** para um dos objetos, mas **abre uma nova conexão TCP em paralelo** para solicitar o outro simultaneamente.

- **Vantagens:** Excelente desempenho. Poupa o RTT de abertura para os objetos que reaproveitam as conexões já abertas, enquanto mantém o paralelismo para acelerar o carregamento global da página.
- **Desvantagens:** Ainda consome múltiplos recursos do servidor (vários *sockets* abertos por cliente) e sofre com os recursos alocados enquanto as conexões são mantidas ociosas.

**Análise de Tempo:** O HTML custa 2 RTT. A 'Img1' reaproveita a conexão (1 RTT), enquanto a 'Img2' exige nova conexão (2 RTT). Sendo paralelas, o tempo final da etapa é o maior entre eles:

$$\text{Tempo Total} \approx 2 \text{ RTT (HTML)} + \max(1 \text{ RTT}, 2 \text{ RTT}) = 4 \text{ RTT}$$

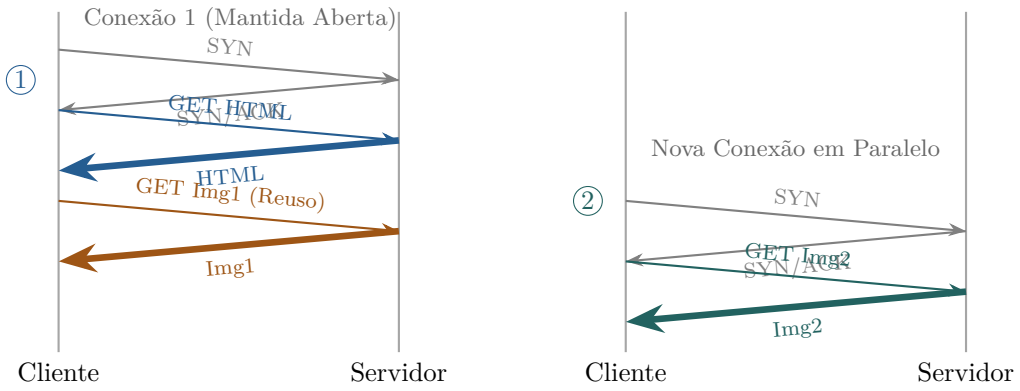



Figure 11: Persistente Com Paralelismo: A Conexão 1 é reaproveitada para a Img1 poupando tempo, enquanto uma Nova Conexão 2 é aberta para baixar a Img2 simultaneamente.

## 9.4 Evolução do HTTP e a técnica de Interleaving

Para mitigar problemas clássicos de travamento de rede (onde a lentidão de um arquivo trava a transmissão dos menores, e.g., Head-of-Line Blocking), os protocolos evoluíram:

- **HTTP/1 (Básico):** Opera sobre **TCP**. As respostas do servidor são enviadas inteiras, de forma *FIFO* (First-In, First-Out). **Não possui** suporte nativo a *interleaving*.
- **HTTP/2:** Opera sobre **TCP**. Introduz o **interleaving** de forma nativa na mesma conexão, permitindo que as respostas venham em ordem mais estratégica.
- **HTTP/3:** Abandona o TCP em favor do **UDP**. Construído com o protocolo **QUIC** e com foco forte em **segurança**, reduz radicalmente a latência da conexão inicial (podendo zerar o RTT inicial).

### 9.4.1 O conceito de Interleaving

O **interleaving** permite que o servidor quebre os arquivos e envie suas partes **intercaladas** através da mesma conexão, não exigindo que um arquivo enorme termine de ser enviado antes de começar o próximo.

**A ilusão de rapidez:** O *interleaving* manda as partes intercaladas (um pouco de HTML, um pouco de Imagem, etc.) para fazer parecer ao usuário que a página está carregando mais rápido, já que o navegador consegue renderizar trechos dos elementos simultaneamente, ao invés de exibir uma página travada.

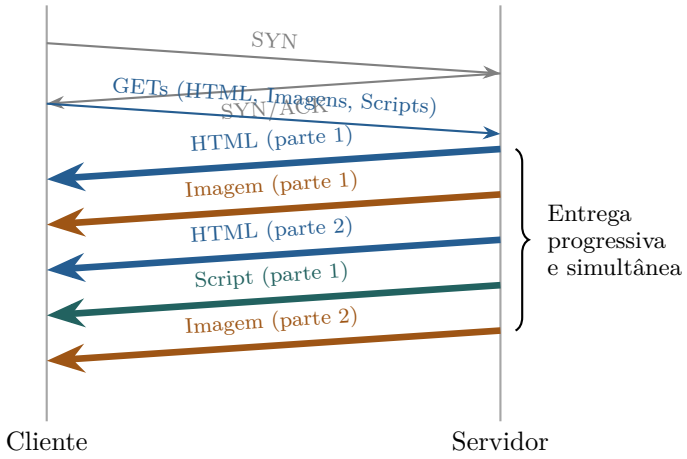



Figure 12: Diagrama esquemático do *interleaving* (HTTP/2). Note que o *handshake* TCP (triângulo cinza no topo) ocorre apenas uma vez, antes da transmissão fatiada.

## 9.5 Questões de revisão

1. Por que a fila $D/D/1$ pode ser estável com $I = 1$, enquanto a $M/M/1$ não pode?
2. O que caracteriza cada posição na notação $A/B/m$? Compare $D/D/1$, $M/D/1$ e $M/M/1$.
3. Por que a espera se escreve como $\overline{W} = \overline{N}_q\overline{S} + \overline{N}_s\overline{S}_R$ e não como $(\overline{N}_q + \overline{N}_s)\overline{S}$ em geral?
4. Por que o segundo momento aparece na expressão de $\overline{S}_R$?
5. Mostre que $\overline{S}_R = \overline{S}$ para serviço exponencial e $\overline{S}_R = \overline{S}/2$ para serviço determinístico.
6. Qual é a interpretação de $\overline{S}_R = \overline{S}$ em termos da propriedade *memoryless*?
7. Dois enlaces têm o mesmo $\overline{S}$ e a mesma taxa de chegada, mas variabilidades diferentes nos tempos de serviço. Qual tem maior espera? Por quê?
8. Obtenha $\overline{W}_{M/D/1}$ a partir da expressão geral e da Lei de Little.
9. Interprete $\pi_0 = 1 - I$. O que significa $\pi_0 = 0{,}2$?
10. Deduza $\lambda = \mu I$ condicionando a vazão nos estados ocupado e ocioso.
11. Em um caminho com $R_c = 5$ Mbit/s, $R_s = 100$ Mbit/s e um enlace compartilhado de $R = 20$ Mbit/s dividido por 10 conexões, qual é a vazão de cada conexão? Qual é o gargalo?
12. Como um cache próximo ao cliente altera a intensidade de tráfego no enlace de acesso?
13. Qual é a função do campo TTL e como o traceroute a utiliza?
14. Por que os três *probes* de um mesmo salto do traceroute costumam apresentar tempos diferentes?
15. Quais são as quatro formas de comunicação entre processos? Qual delas funciona entre máquinas distintas e por quê?
16. O que é um socket, quem o implementa e onde ele reside?
17. Qual é a diferença de papel entre o endereço IP e o número de porta?
18. Em que sentido o TCP é *heavyweight* e o UDP é *lightweight*? Sobre qual deles o QUIC opera?

**AULA 10 | 11/09/2026 – sexta-feira**

# Cookies, cache Web, HTTP/2, correio eletrônico e DNS

**Cobertura:** Capítulo 2, Slides 43 a 104.

## 10.1 Cookies e privacidade

O HTTP é um protocolo sem estado: cada requisição é tratada de forma independente, e o servidor não guarda memória das requisições anteriores. Isso simplifica o projeto do servidor, mas impede funcionalidades que dependem de continuidade, como um carrinho de compras ou uma sessão autenticada.

Os *cookies* resolvem isso reintroduzindo estado por fora do protocolo. São quatro componentes:

1. uma linha de cabeçalho na mensagem de resposta HTTP (`Set-cookie`);
2. uma linha de cabeçalho na mensagem de requisição HTTP (`Cookie`);
3. um arquivo mantido no computador do usuário e gerenciado pelo navegador;
4. um banco de dados no site.

O funcionamento é direto. Na primeira visita, o site gera um número de identificação, cria uma entrada no seu banco de dados indexada por esse número e o devolve no cabeçalho `Set-cookie`. O navegador armazena o número. Em toda requisição subsequente àquele site, o navegador envia o mesmo número no cabeçalho `Cookie`, e o servidor consegue associar aquela requisição a todas as anteriores.

Os usos legítimos são vários: autorização, carrinho de compras, recomendações, estado de sessão.

### 10.1.1 A questão da privacidade

O mesmo mecanismo permite rastreamento. O ponto central é que os objetos de uma página não precisam vir do mesmo servidor que a página. Quando uma página inclui um recurso hospedado por terceiros, o navegador faz uma requisição a esse terceiro e envia a ele o cookie correspondente.

Se o mesmo terceiro estiver presente em muitos sites diferentes, ele recebe uma requisição sempre que o usuário visita qualquer um deles, e todas essas requisições carregam o mesmo identificador. Com isso, é possível reconstruir o histórico de navegação do usuário através de sites não relacionados entre si, e associá-lo a um perfil.

Note que o site visitado não precisa informar nada ao terceiro: a simples inclusão do recurso já produz a requisição. O usuário também não é consultado, porque o envio do cookie é automático.

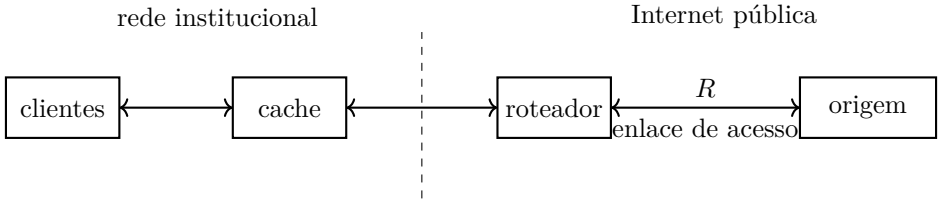
As contramedidas usuais são o bloqueio de cookies de terceiros pelo navegador, a limitação do tempo de vida dos cookies e o uso de modos de navegação que descartam o armazenamento ao fim da sessão.

## 10.2 Cache Web

Um cache Web, também chamado servidor *proxy*, é um elemento de rede que atende requisições HTTP em nome do servidor de origem. O navegador é configurado para dirigir todas as requisições ao cache.

O fluxo é o seguinte: o navegador abre uma conexão TCP com o cache e envia a requisição. Se o cache tiver uma cópia do objeto, ele a devolve imediatamente. Se não tiver, abre uma conexão TCP com o servidor de origem, requisita o objeto, guarda uma cópia local e devolve outra ao navegador.

Um cache é simultaneamente servidor e cliente: servidor em relação aos navegadores que atende, cliente em relação aos servidores de origem que consulta.




Há duas motivações. A primeira é reduzir o tempo de resposta percebido pelo cliente, já que o cache está próximo e ligado por um enlace rápido. A segunda, menos óbvia e mais importante, é reduzir o tráfego no enlace de acesso da instituição, o que evita a necessidade de ampliar esse enlace.

### 10.2.1 Exemplo numérico

Considere uma rede institucional com as seguintes características:

- tamanho médio do objeto: $L = 100$ kbits;
- taxa média de requisições: $a = 15$ requisições/s;
- enlace de acesso: $R = 1{,}54$ Mbit/s;
- atraso médio entre o roteador de acesso e os servidores de origem: 2 s;
- LAN institucional de alta velocidade, com atraso desprezível.

A taxa média de dados entregue aos navegadores é

$$La = 100\,000 \times 15 = 1{,}50 \text{ Mbit/s}.$$

#### 10.2.1.1 Situação inicial

A intensidade de tráfego no enlace de acesso é

$$I = \frac{La}{R} = \frac{1{,}50}{1{,}54} \approx 0{,}974.$$

O número médio de objetos no enlace é

$$\overline{N} = \frac{I}{1 - I} \approx \frac{0{,}974}{0{,}026} \approx 37{,}5,$$

e o tempo médio no enlace é

$$\overline{T} = \frac{\overline{N}}{a} \approx \frac{37{,}5}{15} \approx 2{,}5 \text{ s}.$$

O atraso total fica em torno de 4,5 s: dois segundos na Internet pública e dois e meio na fila do enlace de acesso. O problema é a proximidade de $I$ a 1: nessa região, pequenas variações na carga produzem variações enormes no atraso.

#### 10.2.1.2 Primeira solução: ampliar o enlace

Elevando o enlace de acesso para $R = 154$ Mbit/s,

$$I = \frac{1{,}50}{154} \approx 0{,}0097,$$

o que torna o atraso de fila desprezível, da ordem de fração de milissegundo. O atraso total cai para aproximadamente 2,01 s, dominado pela travessia da Internet pública.

A solução funciona, mas é cara: multiplica por cem a capacidade contratada do enlace.

#### 10.2.1.3 Segunda solução: instalar um cache

Suponha uma taxa de acerto de 0,4. Isso significa que 40% das requisições são atendidas localmente, em torno de 10 ms, e apenas 60% atravessam o enlace de acesso.

O tráfego no enlace passa a ser

$$0{,}6 \times 1{,}50 = 0{,}90 \text{ Mbit/s},$$

e a intensidade de tráfego cai para

$$I = \frac{0{,}90}{1{,}54} \approx 0{,}584.$$

Agora

$$\overline{N} = \frac{0{,}584}{0{,}416} \approx 1{,}41, \qquad \overline{T} = \frac{1{,}41}{15} \approx 0{,}094 \text{ s}.$$

O atraso médio total é a média ponderada dos dois caminhos:

$$\begin{aligned} \overline{T}_{\text{total}} &= 0{,}4 \times (0{,}01) + 0{,}6 \times (2 + 0{,}094) && (44)\\ &\approx 0{,}004 + 1{,}256 && (45)\\ &\approx 1{,}26 \text{ s}. && (46) \end{aligned}$$

O resultado é melhor do que o da primeira solução, a um custo muito menor. Esse é o argumento central em favor dos caches: eles atacam a intensidade de tráfego no gargalo, e não a capacidade do gargalo.

### 10.2.2 GET condicional

O cache introduz um problema novo: a cópia armazenada pode estar desatualizada. O mecanismo que resolve isso é o GET condicional.

Ao guardar um objeto, o cache registra também a data da última modificação informada pelo servidor. Em uma requisição posterior, o cache inclui o cabeçalho

`If-modified-since:` ⟨data⟩.

Se o objeto não mudou desde aquela data, o servidor responde com o código `304 Not Modified` e um corpo vazio, o que economiza a transferência completa. Se mudou, responde normalmente com o objeto atualizado.

## 10.3 HTTP/2 e o bloqueio de cabeça de fila

No HTTP/1.1 com conexão persistente, o cliente pode enviar várias requisições seguidas, mas o servidor devolve os objetos na ordem em que foram pedidos, um após o outro, sobre a mesma conexão TCP.

Isso cria o bloqueio de cabeça de fila, ou *head-of-line blocking*: um objeto grande à frente atrasa todos os objetos pequenos atrás dele.

### 10.3.1 Exemplo

Um cliente requisita um objeto grande, como um arquivo de vídeo, e três objetos pequenos. Suponha que o objeto grande leve 10 unidades de tempo para ser transmitido e cada objeto pequeno leve 1.

Se o objeto grande for entregue primeiro, os instantes de conclusão são

$$10, \quad 11, \quad 12, \quad 13,$$

com média

$$\frac{10 + 11 + 12 + 13}{4} = 11{,}5.$$

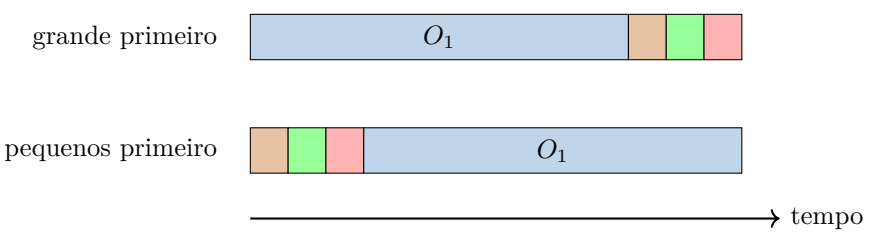
Se os objetos pequenos forem entregues primeiro, os instantes passam a ser

$$1, \quad 2, \quad 3, \quad 13,$$

com média

$$\frac{1 + 2 + 3 + 13}{4} = 4{,}75.$$

O objeto grande termina no mesmo instante nos dois casos, mas o tempo médio de conclusão cai para menos da metade. Nenhuma capacidade foi adicionada: apenas mudou a ordem de atendimento.



### 10.3.2 A solução do HTTP/2

O HTTP/2 divide cada objeto em quadros de tamanho reduzido e intercala os quadros de objetos diferentes na conexão. Com isso, os objetos progridem em paralelo e um objeto grande deixa de monopolizar a conexão. O cliente pode ainda indicar prioridades entre os objetos.

Resta uma limitação: como tudo ainda trafega sobre uma única conexão TCP, a perda de um segmento bloqueia a entrega de todos os fluxos até a retransmissão. O HTTP/3 ataca esse ponto residual ao operar sobre QUIC, que roda sobre UDP e mantém os fluxos independentes.

## 10.4 Correio eletrônico

O sistema de correio eletrônico tem três componentes principais:

- agentes de usuário;
- servidores de correio;
- o protocolo SMTP, *simple mail transfer protocol*.

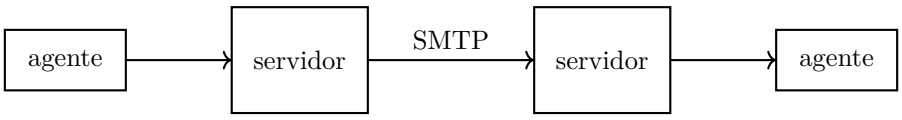
### 10.4.1 Agente de usuário

O agente de usuário é o programa de leitura e composição de mensagens. Ele permite compor, editar e ler mensagens, e as mensagens enviadas e recebidas ficam armazenadas no servidor, não no agente.

### 10.4.2 Servidor de correio

Cada servidor mantém, para cada usuário, uma caixa postal com as mensagens recebidas, e mantém também uma fila de mensagens de saída aguardando envio.

Se a entrega a um servidor remoto falhar, a mensagem permanece na fila e novas tentativas são feitas periodicamente.




### 10.4.3 SMTP

O SMTP usa TCP na porta 25 e transfere a mensagem diretamente do servidor de origem para o servidor de destino, sem servidores intermediários.

A transferência tem três fases: apresentação, transferência das mensagens e encerramento. A conexão é persistente, de modo que várias mensagens podem ser enviadas na mesma conexão.

Uma distinção importante: o SMTP é um protocolo de empurrar, pois quem inicia a conexão é quem tem a mensagem para entregar. Já o acesso à própria caixa postal é feito por protocolos de puxar, como IMAP ou uma interface Web.

## 10.5 DNS

### 10.5.1 O problema

Pessoas são identificadas de várias formas: nome, número de documento, passaporte. Hospedeiros e roteadores da Internet também têm dois identificadores:

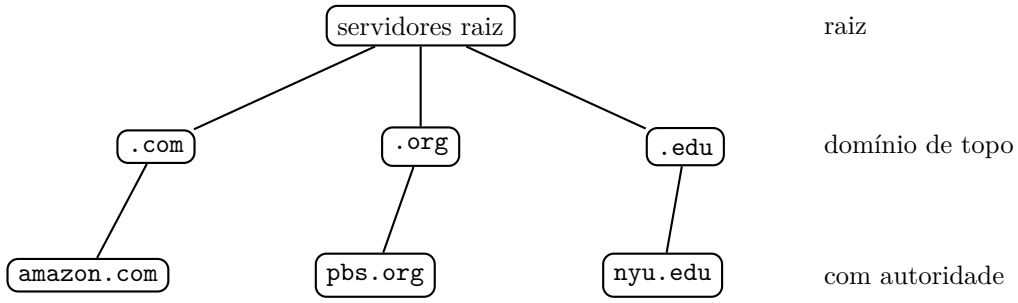
- o endereço IP, de 32 bits, usado para endereçar datagramas;
- o nome, como `cs.umass.edu`, usado por pessoas.

O primeiro é conveniente para as máquinas, o segundo para os usuários. O DNS, *domain name system*, é o serviço que faz o mapeamento entre eles.

Além da tradução de nomes, o DNS oferece apelidos de hospedeiro, apelidos de servidor de correio e distribuição de carga, esta última associando vários endereços IP a um mesmo nome.

### 10.5.2 Base de dados distribuída e hierárquica

Um único servidor centralizado seria inviável: teria um ponto único de falha, concentraria todo o volume de tráfego, ficaria distante da maioria dos clientes e seria impossível de manter. Por isso o DNS é organizado como uma hierarquia de servidores.



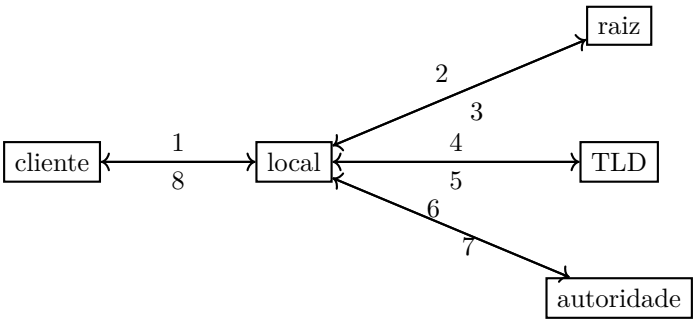

- **Servidores raiz**: conhecem os servidores responsáveis por cada domínio de topo;
- **Servidores de domínio de topo (TLD)**: responsáveis por `.com`, `.org`, `.edu`, e pelos domínios de país;
- **Servidores com autoridade**: guardam os registros oficiais das organizações, mantidos por elas próprias ou por um provedor.

Há ainda um servidor que não pertence à hierarquia mas é o primeiro a ser consultado: o servidor local, configurado pelo provedor de acesso ou pela instituição. O endereço `8.8.8.8` é um exemplo de servidor local público de uso geral.

### 10.5.3 Consulta iterativa e consulta recursiva

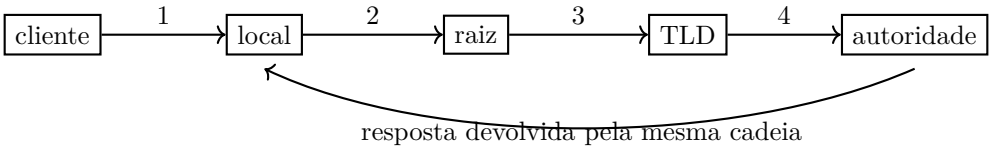
Suponha que um cliente queira o endereço IP de `amazon.com`. Em primeira aproximação, ele consulta um servidor raiz para descobrir o servidor `.com`, e em seguida consulta o servidor `.com` para descobrir o servidor de `amazon.com`. O que muda entre os dois modos é quem carrega o trabalho de percorrer essa cadeia.

Na consulta iterativa, cada servidor contactado responde com a referência do próximo servidor a consultar, e quem percorre a cadeia é o servidor local.




consulta iterativa

Na consulta recursiva, o servidor contactado assume a responsabilidade de obter a resposta completa e devolvê-la pronta a quem perguntou. A cadeia é percorrida servidor a servidor, e cada um responde apenas ao anterior.




consulta recursiva

A diferença prática está em onde fica a carga. A consulta recursiva transfere o trabalho para os servidores de nível mais alto, que atenderiam um volume proibitivo de requisições. Por isso, na prática, a consulta do cliente ao seu servidor local costuma ser recursiva, e dali para cima a cadeia costuma ser iterativa.

### 10.5.4 Cache DNS

Assim que um servidor aprende um mapeamento, ele o guarda localmente e o reutiliza nas consultas seguintes, descartando-o após um prazo de validade.

O efeito é grande: os registros dos servidores de domínio de topo costumam estar em cache nos servidores locais, o que faz com que os servidores raiz raramente sejam consultados de fato. O custo é que uma mudança de endereço só se propaga completamente depois que os registros antigos expiram.

### 10.5.5 Registros

A base de dados é formada por registros de recurso, cada um com nome, valor, tipo e prazo de validade. Os tipos principais são:

- **A**: o nome é um nome de hospedeiro e o valor é o endereço IP;
- **NS**: o nome é um domínio e o valor é o servidor com autoridade sobre ele;
- **CNAME**: o valor é o nome canônico correspondente a um apelido;
- **MX**: o valor é o nome do servidor de correio associado ao domínio.

## 10.6 Redes de distribuição de conteúdo

Um cache institucional reduz o tráfego de uma instituição. A generalização desse princípio em escala global é a rede de distribuição de conteúdo, ou CDN.

Uma empresa de CDN instala grupos de servidores geograficamente distribuídos e replica neles o conteúdo dos provedores. Quando um cliente requisita um objeto, ele é direcionado ao grupo mais próximo ou mais adequado, normalmente por meio do próprio DNS: a consulta ao nome do provedor retorna o endereço de um servidor da CDN em vez do endereço do servidor de origem.

O resultado é que a maior parte do tráfego passa a ser local, reduzindo simultaneamente o atraso percebido e a carga nos enlaces de longa distância.

## 10.7 Síntese

$$\boxed{I = \frac{La}{R}, \qquad \overline{N} = \frac{I}{1-I}, \qquad \overline{T} = \frac{\overline{N}}{a}}$$

Com cache de taxa de acerto $p$, o tráfego no enlace de acesso passa a ser $(1-p)La$, e o atraso médio total é

$$\boxed{\overline{T}_{\text{total}} = p\,\overline{T}_{\text{cache}} + (1-p)\left(\overline{T}_{\text{Internet}} + \overline{T}_{\text{acesso}}\right)}.$$

| **Mecanismo** | **Função** |
|---|---|
| Cookie | estado em um protocolo sem estado |
| GET condicional | validar a cópia em cache |
| HTTP/2 | intercalar quadros, mitigar HOL |
| SMTP (porta 25) | entregar mensagem entre servidores |
| IMAP, Web | acessar a própria caixa postal |
| DNS | traduzir nome em endereço IP |
| CDN | aproximar o conteúdo do cliente |

## 10.8 Questões de revisão

1. Por que o HTTP é descrito como um protocolo sem estado, e quais são os quatro componentes que permitem contornar isso?
2. Como um terceiro presente em vários sites consegue reconstruir o histórico de navegação de um usuário?
3. Em que sentido um cache Web é, ao mesmo tempo, cliente e servidor?
4. Quais são as duas motivações para instalar um cache Web? Qual delas justifica o investimento do ponto de vista da instituição?
5. Com $L = 100$ kbits, $a = 15$ requisições/s e $R = 1{,}54$ Mbit/s, calcule $I$, $\overline{N}$ e $\overline{T}$.
6. Repita o cálculo anterior com $R = 154$ Mbit/s. Por que essa solução é considerada cara?
7. Com uma taxa de acerto de 0,4, recalcule a intensidade de tráfego no enlace de acesso e o atraso médio total.

8. Por que o cache é mais eficaz do que a ampliação do enlace, mesmo reduzindo o tráfego em apenas 40%?
9. O que é o GET condicional e qual código de resposta ele pode produzir?
10. Explique o bloqueio de cabeça de fila no HTTP/1.1.
11. Com um objeto de 10 unidades de tempo e três de 1 unidade, compare o tempo médio de conclusão nas duas ordens de atendimento. Por que o objeto grande termina no mesmo instante nos dois casos?
12. Como o HTTP/2 mitiga o problema, e qual limitação permanece?
13. Quais são os três componentes principais do correio eletrônico?
14. Por que o SMTP é classificado como protocolo de empurrar, enquanto o acesso à caixa postal usa protocolos de puxar?
15. Por que um único servidor DNS centralizado seria inviável?
16. Diferencie consulta iterativa de consulta recursiva. Qual é usada entre o cliente e o servidor local, e por quê?
17. Por que os servidores raiz são consultados com pouca frequência na prática?
18. Qual é a desvantagem do cache DNS quando um endereço muda?
19. Para que serve cada um dos tipos de registro A, NS, CNAME e MX?
20. Como uma CDN utiliza o DNS para direcionar o cliente?

**AULA 11 | 16/09/2026 – quarta-feira**

# DNS, CDN e distribuição P2P

**Cobertura:** fechamento de DNS, redes de distribuição de conteúdo (CDNs) e comparação inicial entre cliente-servidor e P2P.

## 11.1 Protocolos sem estado

Tanto HTTP quanto DNS são, em essência, protocolos *stateless*: cada requisição deve conter informação suficiente para ser respondida sem depender do histórico das requisições anteriores.

No HTTP, cookies e sessões ajudam a reconstruir estado na aplicação. No DNS, o servidor responde consultas por nomes e registros sem precisar manter uma conversa aberta com o cliente.

Ser sem estado simplifica o servidor e melhora a robustez. O preço é que informação de contexto precisa aparecer na própria requisição, ou ser mantida por mecanismos auxiliares, como cache, cookies ou sessões.

## 11.2 DNS: consulta iterativa, recursiva e cache

Na consulta iterativa, o servidor local pergunta a vários servidores da hierarquia DNS: raiz, domínio de topo e servidor autoritativo. Cada resposta indica o próximo passo.

Na consulta recursiva, o servidor consultado assume a responsabilidade de obter a resposta completa. Na prática, a consulta do cliente ao servidor local costuma ser recursiva; a partir do servidor local, a resolução costuma ser iterativa.

O cache DNS evita repetir consultas. Depois que o servidor local aprende uma resposta, ele a reutiliza enquanto o TTL do registro não expira.

O cache melhora desempenho e reduz carga nos servidores superiores, mas torna mudanças de endereço menos imediatas: enquanto o TTL antigo não expira, alguns clientes ainda podem receber a informação anterior.

## 11.3 CDN

Uma CDN replica conteúdo em servidores distribuídos geograficamente. Em vez de todo cliente buscar o objeto no servidor de origem, o cliente é direcionado a um servidor próximo ou conveniente da CDN.

O DNS frequentemente participa desse redirecionamento: a resolução do nome pode retornar um endereço associado à CDN, e não diretamente ao servidor original.

A CDN aplica a mesma ideia do cache em escala global: aproximar conteúdo dos usuários para reduzir atraso, tráfego em enlaces longos e carga no servidor de origem.

## 11.4 Cliente-servidor e P2P

No modelo cliente-servidor, um servidor central envia dados aos clientes. Se há muitos clientes, a carga de envio fica concentrada no servidor.

Se o arquivo tem tamanho $F$, a taxa de upload do servidor é $u_s$ e existem $N$ clientes, então o servidor precisa transmitir pelo menos $NF$ bits ao todo. Assim, um limite inferior para o tempo de distribuição é

$$D_{\text{CS}} \geq \frac{NF}{u_s}.$$

Logo, quando $N$ cresce, o tempo tende a crescer linearmente se o gargalo é o upload do servidor:

$$D_{\text{CS}} \in O(N).$$

No modelo P2P, os clientes também enviam partes do arquivo entre si. Assim, cada novo participante traz demanda, mas também traz capacidade de upload.

$$D_{\text{P2P}} \geq \max\left\{\frac{F}{u_s}, \frac{F}{d_{\min}}, \frac{NF}{u_s + \sum_{i=1}^{N} u_i}\right\}.$$

Quando os pares contribuem com upload, o sistema pode escalar melhor que no modelo puramente cliente-servidor.

## 11.5 BitTorrent

No BitTorrent, o arquivo é dividido em pedaços. Um conjunto de pares que trocam pedaços do mesmo arquivo é chamado *swarm*. Um *tracker*, ou mecanismo equivalente, ajuda um par a encontrar outros participantes.

Dois princípios são importantes:

- **rarest first**: priorizar pedaços raros, para evitar que uma parte do arquivo desapareça do sistema;
- **tit-for-tat**: favorecer pares que também enviam dados, para estimular cooperação.

Se todos baixam os mesmos pedaços primeiro, pode surgir a síndrome do pedaço faltante: muitos pares têm partes iguais do arquivo, mas falta uma parte rara para completar o download.

**AULA 12 | 18/09/2026 – sexta-feira**

# Sockets, UDP e atraso de acesso

**Cobertura:** interface de sockets, diferença entre cliente e servidor, início de multiplexação/demultiplexação e revisão do modelo de atraso no enlace de acesso.

## 12.1 Sockets

Um socket é a interface pela qual uma aplicação usa a rede. A aplicação não envia bits diretamente ao enlace; ela escreve em um socket, e o sistema operacional usa a pilha de protocolos para entregar os dados.

No lado cliente, a sequência típica é:

$$\texttt{socket} \longrightarrow \texttt{connect} \longrightarrow \texttt{send/recv} \longrightarrow \texttt{close}.$$

No lado servidor, a sequência típica é:

$$\texttt{socket} \longrightarrow \texttt{bind} \longrightarrow \texttt{listen} \longrightarrow \texttt{accept} \longrightarrow \texttt{recv/send} \longrightarrow \texttt{close}.$$

O cliente normalmente inicia a conexão. O servidor precisa antes escolher um endereço e uma porta, ficar escutando e aceitar conexões dos clientes.

## 12.2 Portas, multiplexação e demultiplexação

A camada de transporte usa portas para entregar os dados ao processo correto. Várias aplicações podem usar a rede ao mesmo tempo; a camada de transporte multiplexa esses fluxos na origem e demultiplexa no destino.

No UDP, a identificação básica envolve endereço IP e porta. O UDP não cria uma conexão confiável: ele envia datagramas, sem garantia de entrega, ordem ou controle de congestionamento.

UDP é simples e leve, mas a aplicação precisa tolerar perdas ou implementar por conta própria aquilo que for necessário, como retransmissão, controle de ordem ou temporização.

## 12.3 Atraso no enlace de acesso

Considere objetos de tamanho médio $\bar{X}$, chegando a uma taxa média $\lambda$, por um enlace com capacidade normalizada de serviço $\mu$. A intensidade de tráfego é

$$I = \frac{\lambda}{\mu}.$$

Quando $I < 1$, o sistema é estável. Quando $I$ se aproxima de 1, pequenas variações na chegada ou no tamanho dos objetos produzem grande aumento no tempo de espera. Uma forma simples de lembrar esse comportamento é

$$\overline{N} = \frac{I}{1 - I}.$$

Se $I \geq 1$, a fila tende a crescer sem limite:

$$I \geq 1 \quad \Longrightarrow \quad \overline{N} \to \infty.$$

A capacidade média precisa ser maior que a demanda média. Mas não basta estar um pouco abaixo de 1: quanto mais perto de 1, maior o atraso esperado.

## 12.4 Acesso, Internet e tempo total

Em muitos exemplos do capítulo de aplicação, o atraso percebido pelo usuário é decomposto em duas partes:

$$\overline{T}_{\text{total}} = \overline{T}_{\text{Internet}} + \overline{T}_{\text{acesso}}.$$

O enlace de acesso pode dominar o atraso quando sua utilização está alta. Por isso, cache e CDN são úteis: eles reduzem o número de objetos que precisam atravessar o enlace de acesso ou a Internet distante.

**AULA 13 | 23/09/2026 – quarta-feira**

# Revisão: P2P versus cliente-servidor

**Cobertura:** revisão para consolidar DNS, CDN, sockets, UDP e a comparação entre distribuição cliente-servidor e P2P.

## 13.1 Distribuição de um arquivo

Suponha um arquivo de tamanho $F$ que deve ser entregue a $N$ clientes. No modelo cliente-servidor, o servidor precisa enviar uma cópia para cada cliente. Mesmo que os clientes tenham bons enlaces de download, o upload do servidor pode se tornar gargalo.

$$D_{\mathrm{CS}} \geq \max\left\{\frac{NF}{u_s}, \frac{F}{d_{\min}}\right\}.$$

No modelo P2P, os clientes recebem e também reenviam partes do arquivo. A capacidade total de upload cresce com o número de participantes:

$$D_{\mathrm{P2P}} \geq \max\left\{\frac{F}{u_s}, \frac{F}{d_{\min}}, \frac{NF}{u_s + \sum_{i=1}^{N} u_i}\right\}.$$

## 13.2 Exemplo de intuição

Se o servidor tem upload fixo $u_s$, aumentar $N$ aumenta a quantidade total de dados que ele precisaria enviar no modelo cliente-servidor. Em P2P, os novos clientes também podem contribuir com upload.

cliente-servidor: demanda cresce, capacidade central fica fixa;

P2P: demanda cresce, mas a capacidade agregada também cresce.

Por isso, em muitos cenários,

$$D_{\mathrm{CS}} \in O(N), \qquad D_{\mathrm{P2P}} \in O(1)$$

como aproximação qualitativa quando cada novo par contribui com capacidade.

## 13.3 Resumo conceitual

| Aspecto | Cliente-servidor | P2P |
|---|---|---|
| Origem dos dados | Servidor central | Servidor inicial e outros pares |
| Escalabilidade | Limitada pelo upload do servidor | Melhora quando os pares contribuem upload |
| Robustez | Depende fortemente do servidor | Pode continuar enquanto houver pares suficientes |
| Complexidade | Mais simples de controlar | Precisa coordenar pedaços, pares e incentivos |
| Exemplo | Web tradicional | BitTorrent |

## 13.4 Questões de revisão

1. Por que HTTP e DNS são chamados de protocolos sem estado?
2. Qual é a diferença entre consulta DNS iterativa e recursiva?
3. Por que o cache DNS reduz a carga nos servidores raiz?
4. Qual é a principal desvantagem do cache quando um registro muda?
5. Como uma CDN usa DNS para aproximar o conteúdo do cliente?
6. Qual é a diferença entre servidor de origem, cache e CDN?
7. Qual é a sequência básica de chamadas de socket no cliente?
8. Qual é a sequência básica de chamadas de socket no servidor?
9. Para que servem portas na camada de transporte?
10. O que o UDP não garante?
11. Por que o atraso cresce muito quando $I$ se aproxima de 1?
12. Por que o modelo P2P pode escalar melhor que o cliente-servidor?
13. O que é a síndrome do pedaço faltante em BitTorrent?
14. Qual é a função das estratégias *rarest first* e *tit-for-tat*?

**AULA 14 | 25/09/2026 – sexta-feira**

## Não houve aula nesse dia pois foi aplicado a P1

**AULA 15 | 30/09/2026 – quarta-feira**

# Introdução a RDT

**Cobertura:** técnicas de confiabilidade (RDT) e técnicas para melhora de desempenho (go back N, selective repeat).

## 15.1 Princípios do RDT

Reliable Data Transfer (RDT) é um protocolo usado para garantir que os dados enviados por um emissor sejam recebidos corretamente pelo destinatário, mesmo quando o canal de comunicação pode apresentar erros/falhas. Nesse sentido, quanto mais tipos de falhas podem ocorrer, mais sofisticado deve ser o RDT.

Repare que reliable (confiável) é diferente de secure (seguro): o primeiro diz respeito a falhas sem intenção de prejudicar a comunicação, o segundo diz respeito a ataques intencionais, realizados por um agente que tenta explorar as vulnerabilidades do sistema.

| **Protocolo** | **Hipóteses sobre a rede** | **O que o protocolo faz** |
|---|---|---|
| RDT 1 | Rede é confiável | Nada |
| RDT 2 | Corrompe pacotes ($E \rightarrow R$) | ACKs e retransmissões |
| RDT 2.1 | Corrompe pacotes ($E \leftrightarrow R$) | Número de sequência (SN) |
| RDT 3 | Perdas | Temporizador (timeout) |

Table 1: Evolução dos protocolos RDT

## 15.2 Interfaces do RDT

O RDT possui 4 interfaces principais:

- **rdt_send(data):** chamada pela aplicação emissora para solicitar o envio de dados.
- **udt_send(packet):** chamada pelo RDT para enviar um pacote através do canal não confiável.
- **rdt_rcv(packet):** chamada quando um pacote chega ao RDT receptor.
- **deliver_data(data):** chamada pelo RDT receptor para entregar os dados à aplicação.